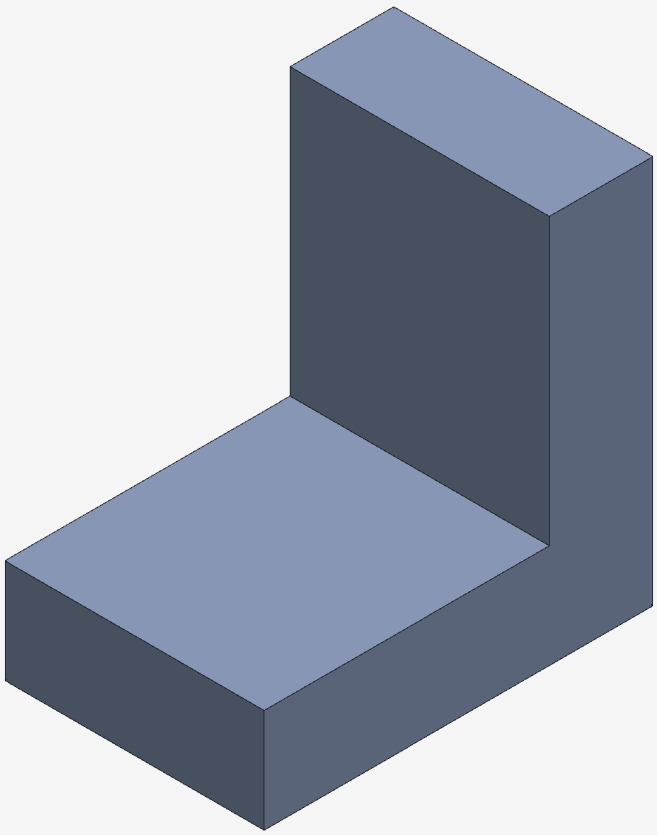


USER MANUAL

Saulitwerx

Open-source parametric 3D CAD

Describes	`main` at commit `b1e6012`
Built	2026-10-28
Status	Pre-alpha: no desktop app yet

How to use this manual

This manual teaches you to use Saulitwerx, an open-source program for designing mechanical parts on a computer. You don't need to know anything about computer-aided design (CAD) to read it. Where a word has a special meaning, the manual explains it the first time it appears, and again in the glossary at the back.

Saulitwerx is young

Saulitwerx is being built in the open, one small step each night. Today it is **pre-alpha**: the parts that do the real geometry work exist and are tested, but there is no desktop app with windows and buttons yet. That changes what this edition of the manual can offer:

- The **Concepts** part is for everyone. It explains how parametric CAD works, with pictures, so you will be ready when the app arrives.
- The **Tasks** part shows what you can already do, by running real commands and small programs. These sections carry a purple **For developers** band, because they need a terminal and some comfort with installing software.
- Nothing in this manual describes a feature as working unless it works on the version named on the cover. Plans are kept apart, in **Coming next** at the back.

How the book is organized

Part	Read it when you want to
I. Getting started	Know what Saulitwerx is, install it, and make your first picture of a part.
II. Concepts	Understand sketches, constraints, dimensions, parameters, features and solids.
III. Tasks	Do one specific thing. Each section is one task, with numbered steps.
IV. Reference	Look up an exact option, default or allowed value.
V. Help	Fix a problem, or understand an error message.
Back matter	Look up a word, see what changed, or find a topic in the index.

Conventions

You see	It means
`swx-qa render`	Text you type exactly, or text the program prints.
Enter	A key on your keyboard.
sketch	A term defined at this spot. It is also in the glossary and the index.
1	A numbered step. Steps start with what to do, then say what you should see.
RESULT	What you have when a task is done.

> **TIP**
>
> Callouts like this one hold shortcuts and good habits.

> **CAUTION**
>
> Cautions warn about a mistake that is easy to make, and how to avoid it.

> **FOR DEVELOPERS**
>
> Sections with this band run programs from a terminal. You can skip them and still follow every concept.

Which version this describes

The cover names the commit of the `main` branch this edition describes, and the date it was built. The latest edition is always at github.com/glitchwizard/saulitwerx, in `docs/manual/`. **What's new**, near the back, lists what each edition added.

Contents

1 Welcome to Saulitwerx 2
- 1.1 What Saulitwerx is 2
- 1.2 What it can do today 2
- 1.3 Who should use it now 3

2 Installing Saulitwerx 4
- 2.1 What gets installed 4
- 2.2 Install the tools 4
- 2.3 Get the source and build it 5
- 2.4 Other systems 6

3 Your first part 7
- 3.1 Draw a sample part 7
- 3.2 Build a plate with a hole 8
- 3.3 What you just did 9

4 How parametric CAD works 11
- 4.1 Solids 11
- 4.2 Units 11
- 4.3 Sketches 12
- 4.4 Constraints 12
- 4.5 Dimensions 13
- 4.6 Degrees of freedom 13
- 4.7 Profiles and regions 14
- 4.8 Parameters 15
- 4.9 Features 15
- 4.10 History 15
- 4.11 Putting it together 16

5 Sketching 18
- 5.1 Solve a sketch 18
- 5.2 How the sample sketch is built 19
- 5.3 Check how fully a sketch is defined 20
- 5.4 Change a dimension 21
- 5.5 Keep a hole centered 21
- 5.6 Draw a slot 23
- 5.7 Find the region to extrude 25
- 5.8 Common mistakes 26

6 Solids and STEP files 27
- 6.1 Make a solid and save it as STEP 27
- 6.2 Combine solids 28
- 6.3 Open a STEP file from another program 28

7 Pictures of parts and sketches 29
- 7.1 Draw a STEP file 29
- 7.2 Choose the size 29
- 7.3 Choose which edges are outlined 29

7.4 Draw a built-in sample ... 30
7.5 Draw a sketch ... 30
8 Parameters and history ... 32
8.1 See what a change rebuilds ... 32
8.2 How the sample is built ... 33
8.3 Name and rename parameters ... 33
8.4 Drive a sketch with parameters ... 34
8.5 Work out an expression ... 35
8.6 Common mistakes ... 37
9 Reference ... 39
9.1 `swx-qa render` ... 39
9.2 `swx-qa sketch` ... 39
9.3 Build options ... 40
9.4 Geometry kernel operations ... 40
9.5 Sketch entities and constraints ... 41
9.6 Solver settings and report ... 42
9.7 Profiles ... 42
9.8 Degree-of-freedom report ... 43
9.9 Parameters ... 44
9.10 Expressions ... 45
9.11 File formats and units ... 45
9.12 Keyboard and mouse ... 45
10 Help ... 47
10.1 Troubleshooting ... 47
10.2 Error messages ... 47
10.3 Questions ... 49
10.4 Known limitations ... 50
Glossary ... 52
What's new ... 54
2026-10-28: dimensions as parameters, expressions, and joined arcs ... 54
2026-10-26: parameter table, and named conflicts ... 54
2026-10-25: more constraints, and profiles ... 54
2026-10-24: degrees of freedom and sketch pictures ... 54
2026-10-22: first edition ... 55
Coming next ... 56
Credits and licenses ... 58
Index ... 59

PART I

Getting started

What Saulitwerx is, what it can do today, how to install it, and a first session that ends with a picture of a real part.

CHAPTER 1

Welcome to Saulitwerx

1.1 What Saulitwerx is

Saulitwerx is a program for designing solid mechanical parts on a computer: brackets, plates, shafts, housings, and anything else you might machine, cast or 3D-print. Programs like this are called *CAD* (computer-aided design). Saulitwerx is free and open source, so anyone can use it, study it, change it, and build it into other software.

Saulitwerx is a *parametric* modeler. You don't sculpt a shape by pushing points around. Instead you describe how to build it, step by step, with measurements: "draw a 40 by 25 mm rectangle, push it up 5 mm, cut a 10 mm hole through the middle." Saulitwerx remembers those steps and their numbers. Change the hole to 12 mm later, and the part rebuilds itself to match. Chapter 4 explains how that works.

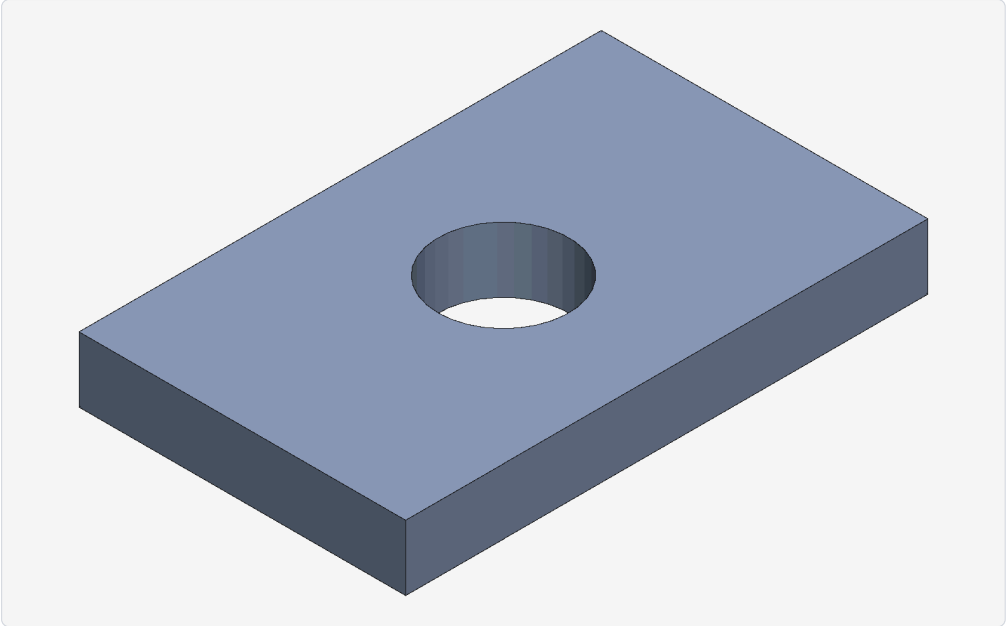


Figure 1.1 A plate with a hole, made by Saulitwerx's geometry engine and drawn by its renderer. You will make this part yourself in Chapter 3.

1.2 What it can do today

Saulitwerx is built in small, tested steps, in the order of a public plan (the *roadmap*). Here is where it stands for this edition.

Area	What exists	Use it
Solids	Make boxes, cylinders and spheres; combine them (add, subtract, keep the overlap); measure volume, surface area and size.	NOW

Area	What exists	Use it
Exchange	Save a solid as a STEP file, and read STEP files back.	NOW
Pictures	Draw a solid or a STEP file to a PNG picture with `swx-qa render`, and a sketch with `swx-qa sketch`.	NOW
Sketching	Lines, circles, arcs and points, with thirteen kinds of constraint (from horizontal and coincident to tangent and symmetric), a solver that moves the shape to satisfy them, a check of how fully a sketch is defined, and the regions a sketch encloses, ready to become solids.	NOW
History	Keep features in order with named parameters, and work out which features a change affects.	NOW
Desktop app	Windows, menus, a 3D view and mouse control.	LATER
Part files	Saving your design with its full history in a Saulitwerx part file.	LATER
Features	Extrude, revolve, fillet, chamfer, hole, shell and more, driven by sketches.	LATER
Assemblies, drawings	Putting parts together, and making 2D shop drawings.	LATER

Everything marked NOW is a building block that programmers can call today. The **Tasks** part shows each one with a real command or a short program you can run. Everything marked LATER is described in **Coming next** at the back, and nowhere else.

1.3 Who should use it now

- **If you are new to CAD,** read Parts I and II. You will learn the ideas every parametric modeler is built on, and you will be ready the day the desktop app arrives.
- **If you are comfortable in a terminal,** install Saulitwerx (Chapter 2) and follow Chapter 3. It takes about 15 minutes and ends with a picture of a part you built.
- **If you write software,** the **For developers** sections show how to call each building block from Rust.

> **NOTE**
>
> Saulitwerx reads and writes only open, documented file formats. It will never open another program's own native files. To bring in a design from elsewhere, export it as STEP first.

CHAPTER 2

Installing Saulitwerx

There is no installer yet. Until the first release, you build Saulitwerx from its source code. That sounds harder than it is: three tools to install, then one command builds everything. Plan on about 10 minutes, most of it waiting for downloads.

YOU WILL	Install the tools Saulitwerx needs, download its source code, build it, and check that it works.
YOU NEED	A computer running Linux (these steps were tested on Ubuntu 24.04), an internet connection, and permission to install software (`sudo`).
TIME	About 10 minutes.

2.1 What gets installed

Tool	Why Saulitwerx needs it	Required?
Git	Downloads the source code and keeps it up to date.	Yes
Rust (1.80 or newer)	The programming language Saulitwerx is written in. Its tool `cargo` builds and runs the program.	Yes
C++ compiler	Builds a small bridge between Saulitwerx and the geometry engine.	For solids and STEP
OpenCascade (OCCT) 7.6 or newer	The open-source geometry engine that does the solid modeling.	For solids and STEP

Without OpenCascade you can still sketch, try the document model, and draw the built-in sample parts. To make your own solids or read and write STEP files, you need it.

2.2 Install the tools

1. Open a terminal. On Ubuntu, press `Ctrl` + `Alt` + `T`. A window with a command prompt appears.
2. Install Git, a C++ compiler and OpenCascade:

   TERMINAL

   ```
   sudo apt-get update
   sudo apt-get install -y git build-essential curl \
      libocct-foundation-dev libocct-modeling-data-dev \
      libocct-modeling-algorithms-dev libocct-visualization-dev \
      libocct-data-exchange-dev
   ```

   Type your password when asked. The terminal prints a list of packages and ends at the prompt again.
3. Install Rust with its official installer, and accept the default choices when it asks:

   TERMINAL

   ```
   curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
   ```

It ends with `Rust is installed now. Great!`

4. Close the terminal and open a new one, so it finds the new tools. Then check them:

```
TERMINAL
cargo --version
```

You should see a version line such as `cargo 1.97.0 (…)`. Any version 1.80 or newer works.

2.3 Get the source and build it

1. Download the source code into a folder named `saulitwerx`, and go into it:

```
TERMINAL
git clone https://github.com/glitchwizard/saulitwerx
cd saulitwerx
```

2. Build Saulitwerx and run its own checks:

```
TERMINAL
cargo test --workspace
```

The first build downloads a few small libraries and takes a minute or two. It ends with several lines that start `test result: ok.` and none that say `FAILED`.

3. Build the picture tool with OpenCascade support:

```
TERMINAL
cargo build --release -p swx-qa --features occt
```

It ends with a line that starts `Finished`. The tool is now at `target/release/swx-qa`.

4. Check that the picture tool runs:

```
TERMINAL
target/release/swx-qa --help
```

You should see its help text, which starts `Usage: swx-qa render`.

RESULT Saulitwerx is built, its tests pass, and the `swx-qa` picture tool is ready. Go on to Chapter 3.

> **TIP** **Staying up to date**
>
> Saulitwerx changes most nights. To get the newest version, go into the `saulitwerx` folder and run `git pull`, then build again.

> **CAUTION** **Commands run from the `saulitwerx` folder**
>
> Every command in this manual assumes your terminal is in the `saulitwerx` folder. If a command says it can't find a file, run `cd saulitwerx` first. Chapter 10 lists other problems and their fixes.

2.4 Other systems

The build is regularly checked only on Ubuntu Linux. Saulitwerx aims to run on macOS and Windows too, and the same steps should work there once Rust and OpenCascade are installed, but they are untested. Without OpenCascade, leave out `--features occt` in step 3.

CHAPTER 3

Your first part

In this tutorial you draw a sample part, then build a real one, a plate with a hole through it, measure it, save it in a standard file format, and draw it. Along the way you will meet the ideas the rest of the manual builds on.

YOU WILL	Draw a built-in sample part, build a plate with a hole, check its measurements, save it as a STEP file, and draw it.
YOU NEED	Saulitwerx built with OpenCascade support (Chapter 2), and a terminal in the `saulitwerx` folder.
TIME	About 15 minutes.

3.1 Draw a sample part

Saulitwerx comes with three sample parts built in: a cube, a cylinder and an L-shaped bracket. They need nothing else, so they are a quick way to check that drawing works.

1. Draw the cube to a picture file named `cube.png`:

   TERMINAL

   ```
   target/release/swx-qa render --fixture cube -o cube.png
   ```

   The tool answers with one line:

   OUTPUT

   ```
   wrote cube.png (800x600, 27669 bytes)
   ```

2. Open `cube.png` in any image viewer. On Ubuntu, type `xdg-open cube.png`. You should see a grey-blue cube on a light grey background, like Figure 3.1.

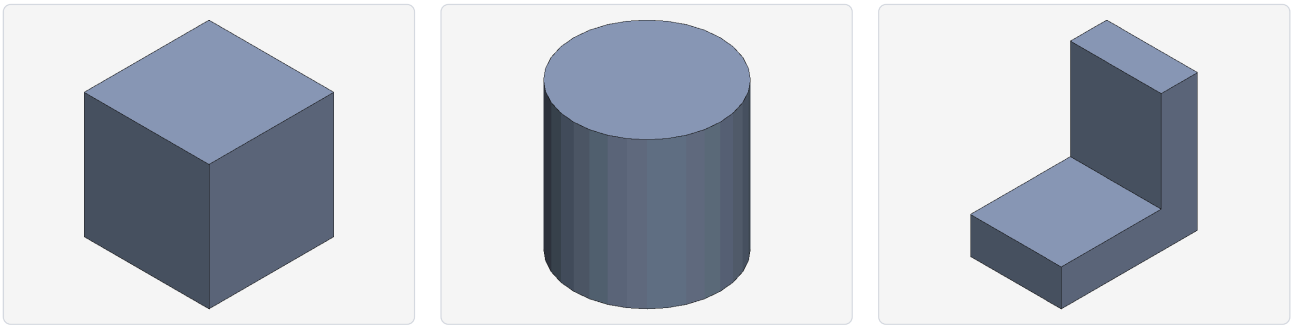


Figure 3.1 The three built-in sample parts: `cube`, `cylinder` and `l-bracket`.

The picture is always drawn from the same corner, looking down at an angle, so that three sides of a box show at once. Each flat face gets one even shade, and sharp edges get a dark outline. This kind of view is called *isometric*.

> **TIP**
>
> Try `--fixture cylinder` and `--fixture l-bracket` too. Add `--size 1600x1200` for a bigger, sharper picture.

3.2 Build a plate with a hole

Now build a real part. Saulitwerx doesn't have a window to draw in yet, so this part comes from a small program that ships with Saulitwerx. The program asks the geometry engine for three things:

1. a *solid* block 40 mm long, 25 mm wide and 5 mm thick;
2. a cylinder 10 mm across, standing where the hole should go, a little taller than the plate;
3. the block **minus** the cylinder. Taking one solid away from another is called a *cut*.

1. Run the program, and tell it to save the part as `plate.step`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example plate_with_hole -- plate.step
   ```

   After a few seconds of building, it prints the part's measurements. The geometry engine also prints a short report about writing the file, framed in rows of `*`; you can ignore it.

   OUTPUT

   ```
   volume        4607.30 mm³
   surface area  2650.00 mm²
   faces         7
   bounding box  [0.0, 0.0, 0.0] to [40.0, 25.0, 5.0]
   …  (the geometry engine's file report)
   wrote plate.step (read back: 4607.30 mm³, 7 faces)
   ```

2. Check the numbers. The block holds $40 \times 25 \times 5 = 5000$ mm³. The hole removes $\pi \times 5^2 \times 5 \approx 392.7$ mm³. $5000 - 392.7 = 4607.3$, which is what the program reports. The part has 7 faces: top, bottom, four sides and the inside of the hole.

3. Draw the part:

   TERMINAL

   ```
   target/release/swx-qa render --step plate.step --size 1600x1000 -o plate.png
   ```

   The tool answers `wrote plate.png (1600x1000, …)`.

4. Open `plate.png`. You should see the plate of Figure 3.2, with the hole showing light background through it.

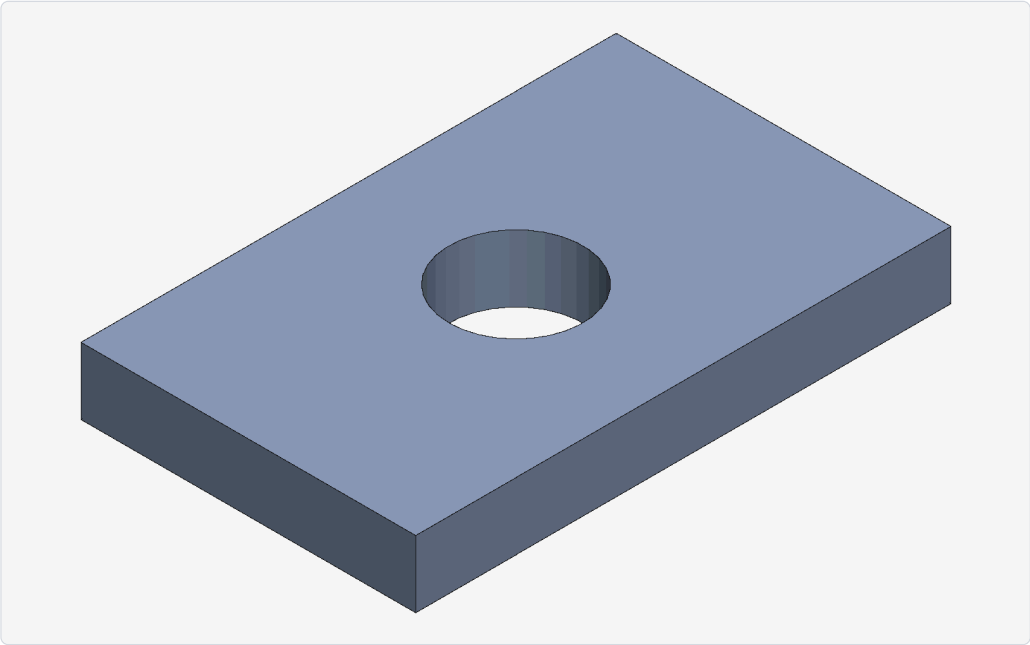


Figure 3.2 The plate you built, drawn from the STEP file the program saved.

RESULT You built a solid part, checked its volume, saved it as `plate.step`, and drew it as `plate.png`.

3.3 What you just did

The file `plate.step` is in the *STEP* format, an international standard (ISO 10303) for exchanging 3D parts. Almost every CAD program can open it, so you can take this plate into another tool, send it to a machine shop, or prepare it for 3D printing.

What you did **not** see yet is the parametric part of Saulitwerx: the numbers 40, 25, 5 and 10 are typed into the program, not stored as named, editable parameters. That comes with the part features and part files on the roadmap. Chapter 4 explains the ideas behind it, and Chapter 8 shows the piece of it that already works.

> **FOR DEVELOPERS**
>
> The program you ran is `crates/swx-kernel/examples/plate_with_hole.rs`. Change `5.0` (the hole radius) in the `make_cylinder` line to `8.0`, run step 1 again, and the volume drops to 3994.69 mm³. Chapter 6 explains each call.

PART II

Concepts

The ideas behind parametric CAD, in plain words. Read this before the task chapters; every procedure later relies on these terms.

CHAPTER 4

How parametric CAD works

Every parametric modeler, Saulitwerx included, is built on a handful of ideas. Once you know them, the commands make sense on their own. This chapter explains each idea in the order you meet it when you design a part. For each one it says what it is, why it matters, and where it stands in Saulitwerx today.

4.1 Solids

A *solid* is a closed 3D shape with an inside and an outside, like a real object. Because Saulitwerx knows what is inside, it can tell you how much material a part holds (its *volume*), how much surface it has to paint (its *surface area*), and how big a box it would fit in (its *bounding box*).

The outside of a solid is made of *faces*: flat or curved patches of surface. Where two faces meet there is an *edge*. The plate from Chapter 3 has 7 faces: top, bottom, four sides, and the round inside of the hole.

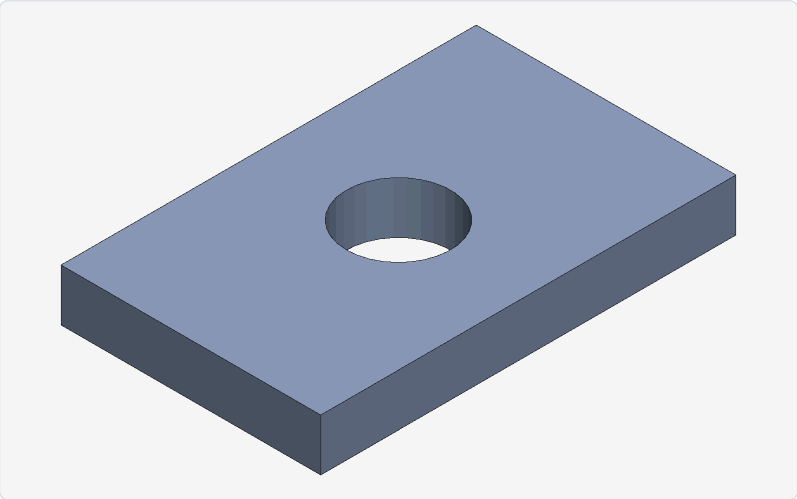


Figure 4.1 A solid. Its faces are shaded; its edges are outlined. Volume 4607.30 mm³, 7 faces.

In Saulitwerx today: the geometry engine makes boxes, cylinders and spheres, and measures volume, surface area, bounding box and face count. Chapter 6 shows how.

4.2 Units

Every length in a part has a *unit*. In Saulitwerx the default is the millimeter (mm), and all of today's building blocks work in millimeters: a box 40 long is 40 mm long, and volumes come out in mm³. Angles are in degrees where you type them, and in radians inside the sketch geometry.

In Saulitwerx today: everything is in millimeters. STEP files are written in millimeters too. Choosing units per part comes with part files.

4.3 Sketches

Most solids start as a flat drawing. A *sketch* is a 2D drawing on a flat plane, made of simple pieces: points, straight lines, circles and arcs. Together these are called sketch *geometry* or *entities*.

A sketch for the plate would be a rectangle (four lines) and a circle. You don't need to draw it accurately. A rough shape is fine, because the next two ideas, constraints and dimensions, make it exact.

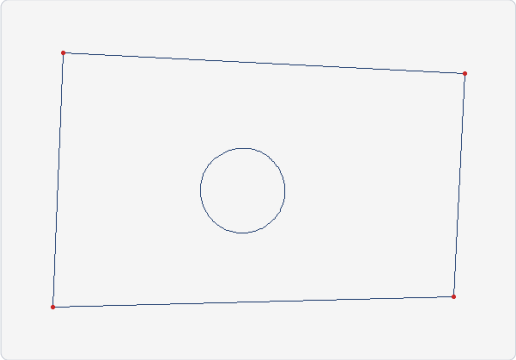


Figure 4.2 A rough sketch of the plate: four lines and a circle, drawn by eye. The red dots are the ends of the lines.

In Saulitwerx today: sketches with points, lines, circles and arcs exist as a building block (Chapter 5). There is no drawing window yet, so sketches are written as a short program, and `swx-qa sketch` draws them to a picture (Section 7.5).

4.4 Constraints

A *constraint* is a rule about the shape of a sketch: "this line is horizontal," "these two corners touch," "this point stays at the origin." Constraints carry the **intent** of a design. If you later change a size, a line that must stay horizontal stays horizontal.

Constraint	Rule it keeps
Coincident	Two points are at the same place. Use it to join the end of one line to the start of the next.
Horizontal	A line runs left–right; or two points are at the same height.
Vertical	A line runs up–down; or two points are directly above each other.
Fixed	A point stays at exact coordinates. Use it to anchor a sketch.
Parallel	Two lines point the same way, like opposite sides of a rectangle.
Perpendicular	Two lines meet at a right angle (90°), like the corner of a rectangle.
Equal	Two lines are the same length, or two circles or arcs are the same size.
Midpoint	A point sits exactly halfway along a line.
Symmetric	Two points are mirror images of each other across a line.
Tangent	A line just touches a circle or arc without cutting into it; or two circles or arcs just touch.

Some constraints are about direction (horizontal, vertical, parallel, perpendicular), some about position (coincident, fixed, midpoint, symmetric), and some about how curves meet (tangent) or match (equal). Parallel and perpendicular are relative: a parallel pair can lean at any angle, as long as both lean the same way. That is why a rectangle usually gets one horizontal side, to say which way is level, and perpendicular and parallel for the rest.

Saulitwerx satisfies constraints with a *solver*: a calculation that nudges every point until all the rules hold at once. You give it a rough drawing and the rules; it gives back the exact shape.

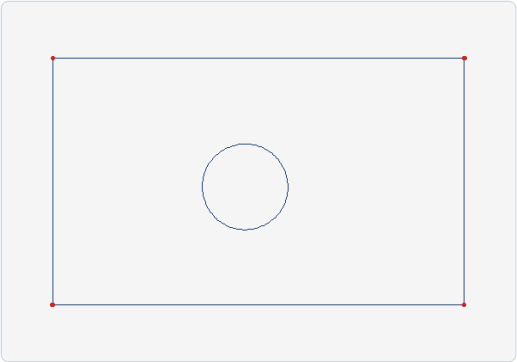


Figure 4.3 The same sketch after the solver applied coincident, horizontal, vertical and fixed. It is now a true rectangle, but its size is still whatever the rough drawing happened to be.

4.5 Dimensions

A *dimension* is a constraint with a number: a length, a distance, a radius or a diameter. “The bottom line is 40 mm long.” “The hole is 10 mm across.” Dimensions fix the size of a sketch, the way the plain constraints above fix its shape.

Dimension	Number it sets
Distance	The straight-line distance between two points, for example the two ends of a line.
Radius	The radius of a circle or arc.
Diameter	The diameter of a circle or arc: twice the radius.

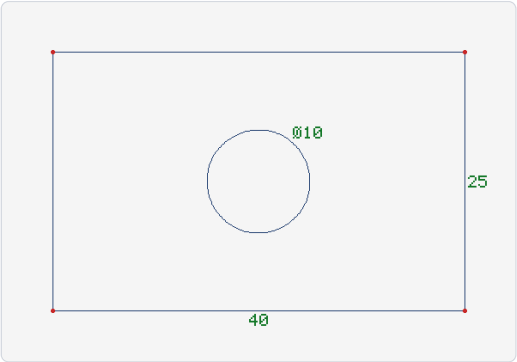


Figure 4.4 With dimensions added, the plate sketch is exactly 40 × 25 mm with a ⌀10 mm hole. Each green number is a dimension.

4.6 Degrees of freedom

Each way a piece of a sketch can still move is called a *degree of freedom*, or DOF for short. A free point on a sheet has two (left–right and up–down). A free line has four (two for each end); a free circle has three (its center’s two, and its size). Each constraint takes some away: fixed takes two from a point, horizontal takes one from a line.

Counting what is left tells you how finished a sketch is. A sketch is in one of four states: *under-constrained*, *fully constrained*, *over-constrained* or *conflicting*.

State	What it means
Under-constrained	Something can still move. Add constraints or dimensions until the count reaches zero.
Fully constrained	Nothing can move, and no rule repeats another. This is the goal.
Over-constrained	At least one rule repeats what other rules already say, but they all agree. The repeated rule is *redundant*: harmless today, but it will fight you as soon as you change one of the sizes it repeats.
Conflicting	Redundant rules that disagree, such as two sides of a rectangle dimensioned 40 and 45 mm. No shape can satisfy them, so the solver stops short.

The plate sketch starts with 19 degrees of freedom: four lines and a circle. Figure 4.3 still has 5 (the rectangle's width and height, and the circle's position and size); Figure 4.4 has none. Section 5.3 shows how to get these numbers.

> **TIP**
>
> Constrain the shape first (coincident, horizontal, vertical), then add dimensions. Then anchor one point with fixed, so the whole sketch can't slide around.

In Saulitwerx today: all of the constraints and dimensions in these tables work, and the solver reports whether it found a shape that meets them. A degree-of-freedom report gives the count and the state, and names each redundant constraint (Section 5.3). Constraints can name every point of a sketch: a point, a line's ends, a circle's or arc's center, and an arc's two ends, so lines and arcs join into closed outlines such as a slot (Section 5.6).

4.7 Profiles and regions

A sketch on its own is just lines. To become a solid, it has to enclose an area that a feature can push into 3D. A closed chain of sketch pieces, where each one ends exactly where the next begins, is a *loop*. The flat area a loop encloses is a *region*, and the regions of a sketch together are its *profile*.

A region can have holes. In the plate sketch the rectangle is one loop and the circle is another, inside it. Saulitwerx works out that the circle is a hole, so the plate's region is the rectangle **minus** the circle: 1000 − 78.54 = 921.46 mm². Pushed up 5 mm, that region is exactly the plate of Chapter 3, hole included, with no separate cut.

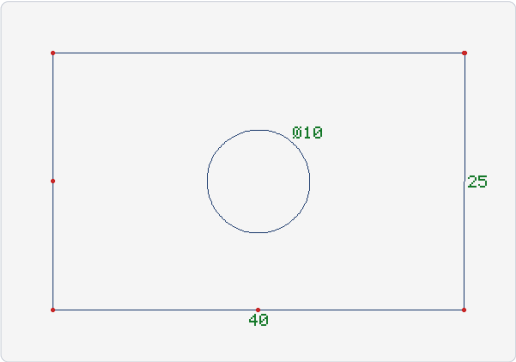


Figure 4.5 Two loops, one region: the rectangle is its outside edge and the circle its hole.

Pieces that don't close a loop, like a line with a free end, are *open edges*. They are left out of every region and listed separately, so you can find the gap. Pieces that cross each other are an error: Saulitwerx won't guess which of the shapes they form you meant.

In Saulitwerx today: profiles are worked out from any sketch of lines, circles and arcs, with holes, nested islands and open edges (Section 5.7). Turning a region into a solid comes with the extrude feature.

4.8 Parameters

A *parameter* is a named number, like `thickness = 5` or `hole_dia = 10`. Features and dimensions refer to parameters by name instead of holding a bare number. Change the parameter, and everything that uses it follows.

This is the "parametric" in parametric CAD. One plate design, with its thickness as a parameter, is really a whole family of plates: set `thickness` to 3, 5 or 8 and rebuild.

Parameters come in two kinds. Each dimension owns a *model parameter*, named `d0`, `d1`, … in the order the dimensions were made. You add *user parameters* yourself, with names you choose. Rename `d0` to `thickness`, and everything that uses it follows.

An *expression* is a small formula with units, such as `d1 / 2 + 5 mm`: half of `d1`, plus 5 millimeters. Saulitwerx checks that the units make sense, so adding a length to an angle is an error, not a silent wrong answer.

In Saulitwerx today: parts keep a parameter table with both kinds, units and comments, and features refer to parameters by name (Section 8.3). A sketch's distance, radius and diameter dimensions can each make their own model parameter and follow it when it changes (Section 8.4). Expressions can be worked out (Section 8.5), but a parameter can't hold one yet: it holds a plain number.

4.9 Features

A *feature* is one step in building a part: a sketch, or something done to the solid, such as pushing a sketch out into 3D or cutting a hole. Each feature records **what** it does and **which** sketch and parameters it uses, not just the shape that resulted.

Figure 4.6 shows the idea of a cut, done directly with the geometry engine: a block, minus a cylinder, gives the plate. A future **extrude** feature will make the block from a rectangle sketch, and a **cut** feature will make the hole from the circle.

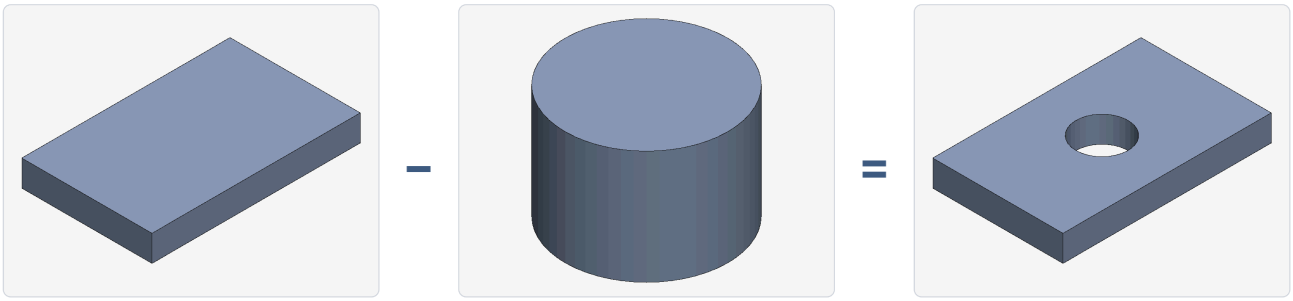


Figure 4.6 A cut takes one solid away from another. Each picture is scaled to fill its frame, so the cylinder looks larger than it is.

There are three ways to combine two solids. Together they are called *Boolean operations*:

- **Union** (join): keep everything that is in either solid.
- **Cut** (subtract): keep the first solid, minus anything the second one overlaps.
- **Intersect**: keep only the part that is in both.

4.10 History

A part's *history*, also called its *feature tree*, is its list of features in the order they were made. Each feature can use only what came before it: a cut needs a solid to cut into, and an extrude needs a sketch to push.

#	Feature	Uses
0	Sketch1	nothing: a rectangle on a plane
1	Extrude1	Sketch1, and the parameter `thickness`
2	Sketch2	nothing: a profile for a pin
3	Revolve1	Sketch2, and the parameter `sweep`

Because the history remembers how each feature was made, Saulitwerx can **rebuild** a part after a change. It doesn't rebuild everything. It works out which features depend on what changed, and redoes only those. Change `thickness`, and only Extrude1 rebuilds; edit Sketch2, and Sketch2 and Revolve1 rebuild.

In Saulitwerx today: the history and the "what needs rebuilding" logic exist (Chapter 8), using the exact example in the table above. Running the rebuild on real geometry comes with the part features.

4.11 Putting it together

Here is how the ideas line up when you design a part, once the app exists:

1. Make a **sketch** on a plane, roughly the shape you want.
2. Add **constraints** to fix its shape, and **dimensions** to fix its size, until no degrees of freedom are left.
3. Turn the sketch into a **solid** with a **feature**, such as extrude.
4. Add more features (cuts, holes, rounded edges) to finish the part. They form its **history**.
5. Change a **parameter** or a dimension later, and the part rebuilds to match.

PART III

Tasks

How to do things, one task per section. Until the desktop app arrives, every task here runs a real command or a small program, and is marked for developers.

CHAPTER 5

Sketching

> **FOR DEVELOPERS**
>
> Until the desktop app's sketch mode arrives, sketches are written as a short Rust program against the `swx-sketch` library. This chapter runs one that ships with Saulitwerx.

5.1 Solve a sketch

The sketch for the plate of Chapter 3 is a 40 × 25 mm rectangle with a 10 mm circle in the middle. In this task you run a program that draws that sketch **roughly**, adds the constraints and dimensions, and asks the solver for the exact shape.

YOU WILL	Run the sample sketch, read the solver's report, and see the rough shape become exact.
YOU NEED	Saulitwerx built (Chapter 2). OpenCascade is not needed.
TIME	5 minutes.

1. Run the sample sketch:

   TERMINAL
   ```
   cargo run -p swx-sketch --example plate_sketch
   ```

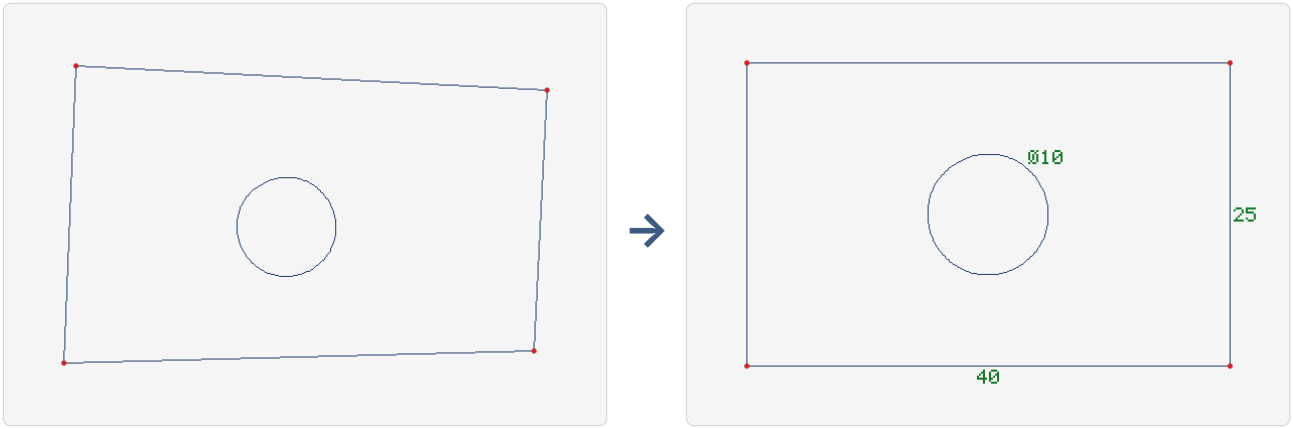
   It prints the solver's report, then each line and the circle:

   OUTPUT
   ```
   solved: true (Converged, 3 iterations, residual 5.3e-14)
   bottom (  0.00,   0.00) -> ( 40.00,   0.00)  length 40.00 mm
   right  ( 40.00,   0.00) -> ( 40.00,  25.00)  length 25.00 mm
   top    ( 40.00,  25.00) -> (  0.00,  25.00)  length 40.00 mm
   left   (  0.00,  25.00) -> (  0.00,   0.00)  length 25.00 mm
   hole   center (20.00, 12.50)  diameter 10.00 mm
   ```

2. Read the first line. `solved: true` and `Converged` mean the solver found a shape that meets every rule. `residual` is how far off it still is, here about 0.00000000000005 mm: exact for any practical purpose.

3. Compare the result with the rough drawing in the program (below). The bottom line was drawn 38 mm long and tilted; it is now exactly 40 mm and level.

RESULT A fully sized, square-cornered sketch: a 40 × 25 mm rectangle with its lower-left corner at the origin, and a 10 mm circle centered at (20, 12.5).



Before: the rough drawing After: solved

Figure 5.1 The sample sketch before and after solving. Section 7.5 shows how to draw pictures like these.

5.2 How the sample sketch is built

The program is `crates/swx-sketch/examples/plate_sketch.rs`. It works in four stages, the same ones you will use in the app. The code below is shortened from the file.

1. Draw roughly. Four lines and a circle, deliberately off-size:

```
let bottom = s.add(Geometry::Line(Line::new(p(0.0, 0.0)?, p(38.0, 1.0)?)?))?;
let right = s.add(Geometry::Line(Line::new(p(38.0, 1.0)?, p(39.0, 22.0)?)?))?;
let top = s.add(Geometry::Line(Line::new(p(39.0, 22.0)?, p(1.0, 24.0)?)?))?;
let left = s.add(Geometry::Line(Line::new(p(1.0, 24.0)?, p(0.0, 0.0)?)?))?;
let hole = s.add(Geometry::Circle(Circle::new(p(18.0, 11.0)?, 4.0)?))?;
```

2. Fix the shape. Join each line's end to the next line's start with **coincident**, then make two lines horizontal and two vertical:

```
for (a, b) in [(bottom, right), (right, top), (top, left), (left, bottom)] {
    s.add_constraint(Constraint::Coincident(PointRef::End(a), PointRef::Start(b)))?;
}
s.add_constraint(Constraint::Horizontal(bottom))?;
s.add_constraint(Constraint::Vertical(left))?;   // and likewise top, right
```

3. Anchor and size it. Pin the lower-left corner to the origin with **fixed**, and set two **distances** and the hole's **diameter**:

```
s.add_constraint(Constraint::Fixed { point: PointRef::Start(bottom), x: 0.0, y: 0.0 })?;
s.add_constraint(Constraint::Distance {
    a: PointRef::Start(bottom), b: PointRef::End(bottom), value: 40.0 })?;
s.add_constraint(Constraint::Diameter { geom: hole, value: 10.0 })?;
```

4. Solve. One call moves everything into place and returns a report:

```
let report = s.solve(SolveOptions::default())?;
```

A `PointRef` names a point the constraint acts on: `Start(line)`, `End(line)`, `Center(circle)`, `ArcStart(arc)`, `ArcEnd(arc)` or `Point(point)`. Section 9.5 lists every entity and constraint with its exact form.

5.3 Check how fully a sketch is defined

A sketch that solves is not necessarily finished: parts of it may still be free to move, or some rules may repeat each other. A degree-of-freedom report (Section 4.6) tells you which. In this task you run a program that builds the plate sketch one group of rules at a time and checks it after each.

YOU WILL	Watch the degrees of freedom of a sketch fall to zero, then see what a repeated or contradicting dimension looks like.
YOU NEED	Saulitwerx built (Chapter 2). OpenCascade is not needed.
TIME	5 minutes.

1. Run the sample:

   TERMINAL

   ```
   cargo run -p swx-sketch --example sketch_dof
   ```

   It prints one line per stage, with the number of equations the rules make, the degrees of freedom left, and the state:

   OUTPUT

   ```
   1 rough drawing            equations  0  dof 19  UnderConstrained
   2 + coincident x4          equations  8  dof 11  UnderConstrained
   3 + horizontal, vertical   equations 12  dof  7  UnderConstrained
   4 + fixed corner           equations 14  dof  5  UnderConstrained
   5 + two distances          equations 16  dof  3  UnderConstrained
     solve: Converged
   6 + hole, then solve       equations 19  dof  0  FullyConstrained
     solve: Converged
   7 + distance top 40        equations 20  dof  0  OverConstrained
                              redundant: distance top 40
     solve: Stagnated
   7 + distance top 45        equations 20  dof  0  Conflicting
                              redundant: distance top 45
   ```

2. Read stages 1 to 6. The four lines and the circle start with 19 degrees of freedom. Each group of rules takes some away, and after the hole is placed and sized none are left: `FullyConstrained`.
3. Read the first stage 7. A third distance, 40 mm on the top line, repeats what the bottom line and the horizontal and vertical rules already say. The solver still succeeds, but the report says `OverConstrained` and names the repeated rule.
4. Read the second stage 7. With 45 mm on the top line instead, no rectangle can meet every rule. The solver stops with `Stagnated`, and the report says `Conflicting` and names the 45 mm distance.

RESULT You can tell an unfinished sketch (`UnderConstrained`, with the number of degrees of freedom still open) from a finished one (`FullyConstrained`), and spot a repeated or contradicting rule.

How the check works

The program is `crates/swx-sketch/examples/sketch_dof.rs`. One call on the sketch makes the report:

```rust
use swx_sketch::dof::DofOptions;

let r = s.dof(DofOptions::default())?;
println!("dof {} {:?}", r.report.dof, r.report.state);
for id in &r.redundant_constraints {
    println!("redundant: {}", names[id.0 as usize]);
}
```

`r.report` holds `dof` and the `state`. `r.redundant_constraints` lists the repeated rules as constraint ids: the number `add_constraint` gave each one, counting from 0 in the order you added them. The sample keeps a list of names in the same order, so `names[id.0]` prints the rule in words. A rule that adds two equations, such as a repeated coincident, is listed once. Section 9.8 lists every field and setting.

> **TIP** **Solve first, then check**
>
> The difference between over-constrained and conflicting is whether the rules are met. Run the check after `solve`: on a rough drawing that hasn't been solved yet, a harmless repeated rule can show as `Conflicting`.

> **CAUTION** **Only the later rule is named**
>
> When two rules repeat each other, the report names the one you added **later**, because the earlier one was still independent when it arrived. The fix may be to delete either one.

5.4 Change a dimension

1. Open `crates/swx-sketch/examples/plate_sketch.rs` in a text editor.
2. Find `value: 40.0` and change it to `value: 60.0`. Save the file.
3. Run the sample again (step 1 above). The bottom and top lines now report `length 60.00 mm`; the right line moves to x = 60. The hole stays at (20, 12.5), because it is fixed there, not centered.
4. Change the value back to `40.0` when you are done.

> **TIP**
>
> The hole stayed put because it is fixed at (20, 12.5). To keep it centered whatever the width, use midpoint instead (Section 5.5).

5.5 Keep a hole centered

A good sketch says what you **mean**. If the hole belongs in the middle of the plate, the sketch should say "in the middle", not "at x = 20". Then a wider plate keeps its hole centered with no extra work. The tool for this is the *midpoint* constraint, which keeps a point exactly halfway along a line.

YOU WILL	Center a hole with midpoint constraints, then change the plate's width and watch the hole follow.
YOU NEED	Saulitwerx built (Chapter 2). OpenCascade is not needed.
TIME	5 minutes.

1. Run the centered-plate sample:

   TERMINAL

   ```
   cargo run -p swx-sketch --example centered_plate
   ```

   It solves the sketch, checks its degrees of freedom, and prints the sizes and the region an extrude would use:

OUTPUT

```
solved: true (Converged), dof 0 FullyConstrained
bottom length 40.00 mm
right  length 25.00 mm
hole   center (20.00, 12.50)  diameter 10.00 mm
regions: 1
  region 0: outer edges 4, holes 1, area 921.46 mm²
open edges: 0
```

2. Check the first line. `FullyConstrained` with `dof 0` means every piece is pinned down, including the hole, even though no rule gives the hole's coordinates.

3. Run it again for a plate 60 mm wide. The two numbers after `--` are the width and the height:

TERMINAL

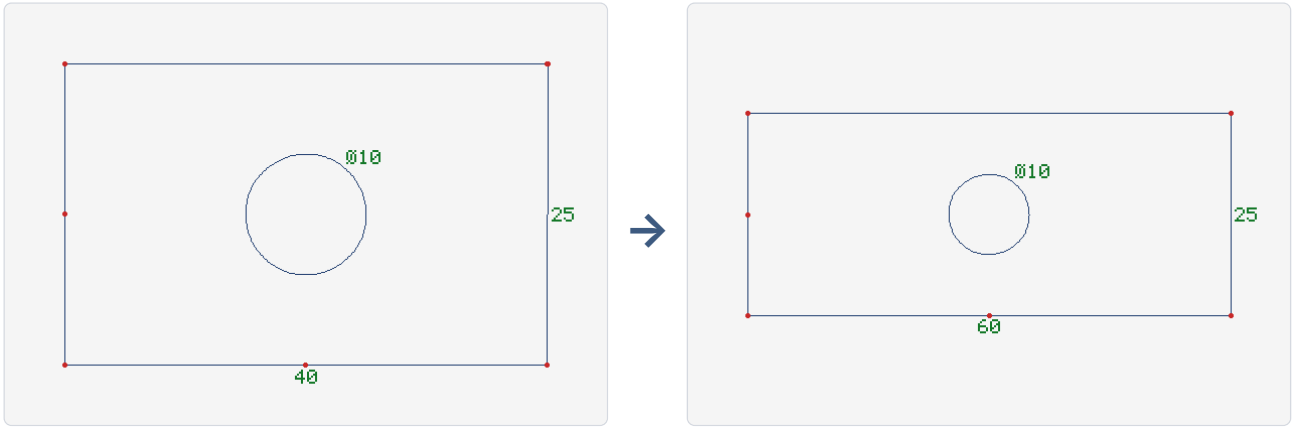
```
cargo run -p swx-sketch --example centered_plate -- 60 25
```

OUTPUT

```
solved: true (Converged), dof 0 FullyConstrained
bottom length 60.00 mm
right  length 25.00 mm
hole   center (30.00, 12.50)  diameter 10.00 mm
regions: 1
  region 0: outer edges 4, holes 1, area 1421.46 mm²
open edges: 0
```

The hole has moved to x = 30, the middle of the new width.

RESULT A plate whose hole stays in the middle whatever its width or height, and a sketch that is fully constrained.



40 mm wide 60 mm wide

Figure 5.2 The same sketch with only the width changed. The red dots at the middle of the bottom and left sides are the helper points that center the hole.

How the centered plate is built

The program is `crates/swx-sketch/examples/centered_plate.rs`. It starts from the same rough drawing as the plate sketch, and changes two things.

Square corners without four direction rules. Only the bottom is horizontal. The left side is *perpendicular* to it, and each side is *parallel* to the one opposite:

```
s.add_constraint(Constraint::Horizontal(bottom))?;
s.add_constraint(Constraint::Perpendicular(bottom, left))?;
```

```
s.add_constraint(Constraint::Parallel(top, bottom))?;
s.add_constraint(Constraint::Parallel(right, left))?;
```

Center the hole with two helper points. A point entity sits at the middle of the bottom side, and another at the middle of the left side. The hole's center is lined up straight above the first and level with the second:

```
let mid_bottom = s.add(Geometry::Point(p(19.0, 0.0)?))?;
let mid_left = s.add(Geometry::Point(p(0.0, 12.0)?))?;
s.add_constraint(Constraint::Midpoint { point: PointRef::Point(mid_bottom), line: bottom })?;
s.add_constraint(Constraint::Midpoint { point: PointRef::Point(mid_left), line: left })?;
s.add_constraint(Constraint::VerticalPoints(
    PointRef::Point(mid_bottom), PointRef::Center(hole)))?;
s.add_constraint(Constraint::HorizontalPoints(
    PointRef::Point(mid_left), PointRef::Center(hole)))?;
```

Helper points are safe to add: they are not part of any outline, so the profile ignores them.

> **TIP** **Why not a diagonal?**
>
> Drawing the rectangle's diagonal and putting the hole's center at its midpoint would also work for the solver. But the diagonal would cut through the hole and the plate, and profiles can't be found from pieces that cross (Section 5.7). Helper points avoid that.

The *symmetric* constraint is the other way to say "centered": `Symmetric { a, b, line }` makes points `a` and `b` mirror images across `line`. It suits pairs, such as two holes placed evenly either side of a center line.

5.6 Draw a slot

A slot is a rounded-end opening, like the hole a bolt slides along. Its outline is two straight sides and two half circles, and the sides must run smoothly into the curves, with no corner. Three constraints say that. Coincident joins each side's ends to the arcs' ends, so the outline closes. The *tangent* constraint makes a line touch a circle or arc without cutting into it, so there is no corner where they meet. The *equal* constraint keeps both ends the same size, so you set the radius only once.

YOU WILL	Close a roughly drawn slot with coincident, tangent and equal constraints, check that it is fully defined, and find its region.
YOU NEED	Saulitwerx built (Chapter 2). OpenCascade is not needed.
TIME	5 minutes.

1. Run the slot sample:

   TERMINAL

   ```
   cargo run -p swx-sketch --example slot_sketch
   ```

   It prints the slot twice: as drawn, then after solving.

OUTPUT

```
1 as drawn
  left   arc center (  0.00,  0.00)  radius 4.00 mm  from   90° to  260°
  right  arc center ( 28.00,  1.00)  radius 6.00 mm  from  -90° to   90°
  bottom (  0.00, -4.00) -> ( 28.00, -5.00)
  regions: 0
  open edges: [0, 1, 2, 3]
2 solved: true
  dof 0 FullyConstrained
  left   arc center (  0.00,  0.00)  radius 5.00 mm  from   90° to  270°
  right  arc center ( 30.00,  0.00)  radius 5.00 mm  from  -90° to   90°
  bottom (  0.00, -5.00) -> ( 30.00, -5.00)
  regions: 1
    outer edges 4, holes 0, area 378.54 mm²
  open edges: []
```

2. Read stage 1. The slot was drawn carelessly on purpose: the ends have radii 4 and 6, the slot is tilted, and the left arc stops at 260°, 10° short of the bottom side (Figure 5.3). Because of that one gap the outline doesn't close, so there is no region and all four pieces are listed as open edges.
3. Read stage 2. Both radii are now 5 mm, the centers are 30 mm apart and level, and the left arc runs the full half circle from 90° to 270°. The solver turned the arc's end until it met the bottom side.
4. Read `dof 0 FullyConstrained`. Every piece of the slot is pinned down: its size, its shape and where it sits. Nothing is left to chance.
5. Read the region. The closed slot encloses one region of 378.54 mm²: a 30 × 10 mm rectangle plus two half circles of radius 5 mm.

RESULT A fully defined 30 mm slot with 5 mm round ends, and one region ready to become a solid.

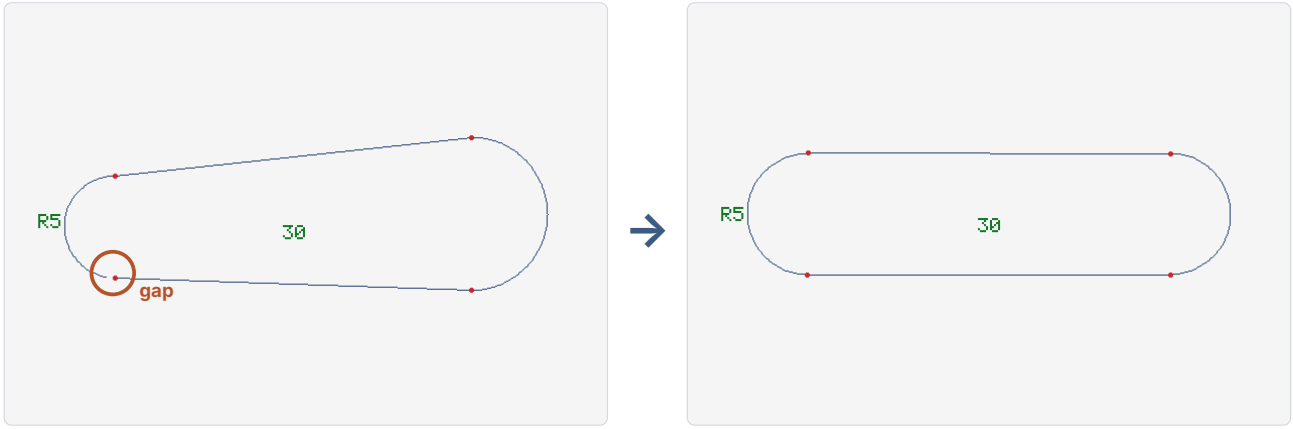


Figure 5.3 The slot before and after solving. As drawn, the ends differ in size, the slot is tilted, and the left arc stops short of the bottom side (ringed). Solved, it is level, with equal 5 mm ends joined all the way round.

How the slot is built

The program is `crates/swx-sketch/examples/slot_sketch.rs`. An arc's two ends are named with `PointRef::ArcStart(arc)` and `PointRef::ArcEnd(arc)`. An arc always runs counterclockwise from its start to its end, so the left arc, drawn from the top (90°) round to the bottom (270°), ends where the bottom side starts:

```
for (a, b) in [
    (PointRef::Start(bottom), PointRef::ArcEnd(left)),
    (PointRef::End(bottom), PointRef::ArcStart(right)),
    (PointRef::Start(top), PointRef::ArcEnd(right)),
```

```
    (PointRef::End(top), PointRef::ArcStart(left)),
] {
    s.add_constraint(Constraint::Coincident(a, b))?;
}
for (line, arc) in [(bottom, left), (bottom, right), (top, left), (top, right)] {
    s.add_constraint(Constraint::Tangent(line, arc))?;
}
s.add_constraint(Constraint::Equal(left, right))?;
s.add_constraint(Constraint::Radius { geom: left, value: 5.0 })?;
```

Then the bottom side is made horizontal, the left center is fixed at the origin, and the centers are set 30 mm apart. That is all the slot needs.

> **TIP** **Which end is which**
>
> Picture walking round the arc counterclockwise, the way the angles grow. Where you set off is `ArcStart`; where you stop is `ArcEnd`. If you join the wrong end, the solver still tries to obey, and you get a different shape from the one you meant. Check the angles in the output, or plot the sketch (Section 7.5).

> **TIP**
>
> A tangent line stays on the side of the circle it started on, and two tangent circles stay touching on the outside or the inside, whichever they were closer to when you added the constraint. Draw roughly the right arrangement and the solver keeps it. When a line's end is joined to an arc's end, tangent there means the line meets the arc square to its radius, which is exactly "no corner".

5.7 Find the region to extrude

Before a sketch can become a solid, Saulitwerx has to know which areas it encloses (Section 4.7). Both samples above end by asking for exactly that. This section shows the call, and how to read the answer.

YOU WILL	Get the regions of a sketch, with their holes and areas, and find the open edges when a sketch doesn't close.
YOU NEED	The two samples above, run once each.
TIME	5 minutes.

1. Look at the end of the centered-plate output (Section 5.5): `regions: 1`, `holes 1`, `area 921.46 mm²`. The rectangle is the region's outside edge and the circle its hole; the area is the rectangle's 1000 mm² minus the hole's 78.54 mm².
2. Look at stage 2 of the slot output (Section 5.6): one region with `outer edges 4` (two lines, two arcs) and no holes.
3. Look at stage 1: `regions: 0` and `open edges: [0, 1, 2, 3]`. The numbers are the pieces in the order they were added to the sketch: 0 and 1 are the arcs, 2 and 3 the lines. When a sketch has no region, the open edges show where to look for the gap.

RESULT You can tell whether a sketch is ready to become a solid, and how much area it encloses.

How it works

One call returns every region and the open edges:

```
use swx_sketch::profile::{profile_report, ProfileOptions};

let profile = profile_report(&s, &ProfileOptions::default())?;
for r in &profile.regions {
    println!("holes {}, area {:.2} mm²", r.holes.len(), r.area());
}
println!("open edges: {:?}", profile.open_edges);
```

`profiles(&s, &options)` returns only the regions. Ends of pieces closer together than a small tolerance count as joined; Section 9.7 lists the setting, every field, and the rules for loops inside loops.

> **CAUTION** **Crossing pieces stop the search**
>
> If two pieces cross, overlap or touch away from their ends, `profile_report` returns an error naming both, such as `geometry 2 and 5 intersect`, and no regions. Saulitwerx doesn't split pieces where they cross. Move or shorten one of them so they meet only end to end.

5.8 Common mistakes

- **A value of zero or less.** Distance, radius and diameter must be greater than zero. Adding `value: 0.0` fails with `value must be > 0, got 0`.
- **The wrong kind of point.** `PointRef::Start` and `End` work only on lines, `ArcStart` and `ArcEnd` only on arcs, and `Center` only on circles and arcs. Using `Center` on a line fails with `geometry 0 has a different kind`.
- **Rules that fight each other.** Two fixed points 30 mm apart with a distance of 40 mm between them can't both hold. The solver then stops with `solved: false` and a reason of `Stagnated`, and moves the points as close to meeting the rules as it can. A degree-of-freedom check then says `Conflicting` and names the clashing constraint (Section 5.3).
- **The wrong kind of entity.** Parallel and perpendicular take two lines. Equal takes two lines or two round pieces (circles and arcs), not one of each. Tangent takes a line and a round piece, or two round pieces, never two lines. Each of these fails with `geometry N has a different kind`.
- **An outline with a gap.** Pieces that look joined but aren't leave no region. Join every corner with coincident, including where a line meets an arc (Section 5.6).
- **Too few rules.** A sketch can solve and still be under-constrained: the solver keeps unconstrained pieces near where you drew them. Check that the degrees of freedom reach zero before you rely on the sizes.

CHAPTER 6

Solids and STEP files

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

The part of a CAD program that does the 3D math is its *geometry kernel*. Saulitwerx talks to its kernel through one interface, the `Kernel` trait, so the rest of the program never depends on which kernel is underneath. Today that kernel is OpenCascade, reached through `OcctKernel`.

6.1 Make a solid and save it as STEP

YOU WILL	Build the plate with a hole, measure it, save it as STEP and read it back.
YOU NEED	Saulitwerx built with OpenCascade (Chapter 2).
TIME	5 minutes.

You already ran this program in Chapter 3. Here is what each part of it does. The file is `crates/swx-kernel/examples/plate_with_hole.rs`.

1. **Make the two starting solids.** A box is given by its lowest corner and its three sizes; a cylinder stands on +Z, given by the center of its base, its radius and its height:

```
let plate = k.make_box([0.0, 0.0, 0.0], [40.0, 25.0, 5.0])?;
let tool  = k.make_cylinder([20.0, 12.5, -1.0], 5.0, 7.0)?;
```

2. **Cut one from the other.** The cylinder starts 1 mm below the plate and ends 1 mm above it, so the hole goes cleanly all the way through:

```
let part = k.cut(&plate, &tool)?;
```

3. **Measure the result** with `volume`, `surface_area`, `face_count` and `bounding_box`. The program prints them; Chapter 3 checks them by hand.

4. **Save and read back.** `export_step` writes the file, in millimeters; `import_step` reads it back as one solid. Matching volume and face count show nothing was lost:

```
k.export_step(&part, &out)?;
let back = k.import_step(&out)?;
```

RESULT A STEP file of the plate, checked by reading it back: `wrote plate.step (read back: 4607.30 mm³, 7 faces)`.

> **TIP Make the tool overlap**
>
> When you cut a hole, make the tool solid a little longer than the part on both sides. A tool that ends exactly on a face can leave a paper-thin skin, or confuse the kernel.

6.2 Combine solids

The kernel has three Boolean operations (Section 4.9). Each takes two solids and returns a new one; the inputs are left unchanged.

Call	Result
`k.union(&a, &b)`	Everything in `a` or `b`.
`k.cut(&a, &b)`	`a` minus whatever `b` overlaps.
`k.intersect(&a, &b)`	Only what is in both.

> **CAUTION**
>
> Sizes must be finite and greater than zero, and positions finite. A box with a zero size is refused before it reaches the kernel, with `invalid input: box dx must be finite and > 0, got 0`.

6.3 Open a STEP file from another program

`import_step` reads any STEP file, and `swx-qa render --step` draws one (Chapter 7). Everything in the file comes in as one shape. Saulitwerx can't edit an imported shape yet; using it as a starting feature is on the roadmap.

CHAPTER 7

Pictures of parts and sketches

`swx-qa render` draws a part to a PNG picture file, and `swx-qa sketch` draws a sketch. They need no graphics card and no window, and the same input always gives the same picture, byte for byte. The project uses them to show its work in reviews; this manual's pictures come from them too.

7.1 Draw a STEP file

YOU WILL Draw any STEP file to a PNG, at the size you want.
YOU NEED `swx-qa` built with OpenCascade (Chapter 2), and a STEP file. `plate.step` from Chapter 3 works.

1. Run `render` with the STEP file, a size and an output name:

   TERMINAL

   ```
   target/release/swx-qa render --step plate.step --size 1600x1000 -o plate.png
   ```

   It answers `wrote plate.png (1600x1000, …)`, with the file size in bytes.

2. Open `plate.png`. The part fills the picture, seen from the same corner as every other render.

RESULT A PNG of the part, ready to share or to put in a document.

The renderer chooses how finely to break curved faces into flat pieces for you: the mesh stays within 0.1 % of the part's overall size of the true surface.

7.2 Choose the size

`--size` takes the width and height in pixels, written `W` `x` `H`, for example `1600x1000`. The default is `800x600`; the largest is `8192x8192`. For a picture you will print, allow about 80 pixels per centimeter (200 per inch) of printed width: a picture printed 10 cm wide needs about 800 pixels.

7.3 Choose which edges are outlined

The renderer outlines the edges where faces meet at a sharp angle. `--crease` sets how sharp, in degrees, from 0 to 180; the default is 30. Where the surface bends by more than this angle from one face to the next, the edge gets a dark line. The edges of a box bend by 90°, so they are always outlined; the many small facets of a curved face bend by only a few degrees, so they are not. A smaller value outlines more edges; a larger value outlines fewer.

> **TIP**
>
> On a curved part that looks striped with lines, raise `--crease` (try 45). If an edge you want to see has no outline, lower it.

7.4 Draw a built-in sample

`--fixture` draws one of three built-in parts instead of a file: `cube` (10 mm), `cylinder` (5 mm radius, 10 mm tall) or `l-bracket` (30 mm legs, 8 mm thick, 20 mm deep). They need no OpenCascade, so they are a quick way to test the tool.

TERMINAL

```
target/release/swx-qa render --fixture l-bracket -o bracket.png
```

Section 9.1 lists every option.

7.5 Draw a sketch

`swx-qa sketch` solves a built-in sketch and draws it from above, with x to the right and y up. Lines, circles and arcs are dark blue; points and the ends of lines are red dots; each dimension's value is written in green beside what it measures.

YOU WILL	Draw a built-in sketch to a PNG.
YOU NEED	`swx-qa` built (Chapter 2). OpenCascade is not needed.
TIME	2 minutes.

1. Run `sketch` with a sample name and an output name:

   TERMINAL

   ```
   target/release/swx-qa sketch --fixture rectangle -o rectangle.png
   ```

   It answers `wrote rectangle.png (800x600, …)`, with the file size in bytes.

2. Open `rectangle.png`. You should see a 40 × 25 mm rectangle, labeled `40` along the bottom and `25` up the right side.

3. Try the other sample, a right triangle with sides of 4 and 3:

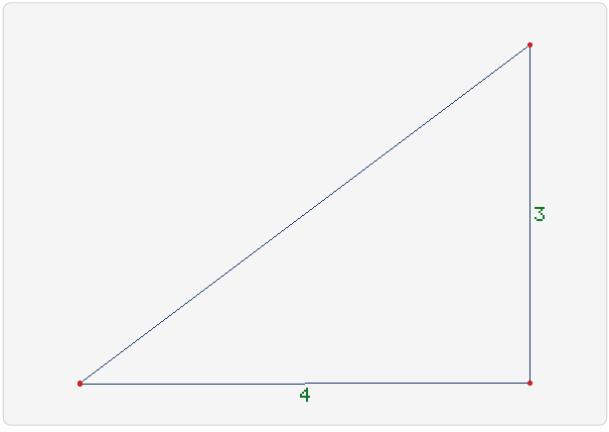
   TERMINAL

   ```
   target/release/swx-qa sketch --fixture triangle-345 -o triangle.png
   ```

RESULT A PNG of a solved sketch with its dimension values (Figure 7.1).



rectangle



triangle-345

Figure 7.1 The two built-in sketches. The triangle's long side has no number: it has no dimension of its own, and comes out 5 because the other two sides are 4 and 3.

`--size` works as it does for parts; the default is `800x600`. The sketch is scaled to fill the picture with a margin, so a small sketch and a large one come out the same size. Labels are drawn at a fixed size in pixels, so they look smaller in a large picture.

FOR DEVELOPERS

To draw your own sketch, call `swx_qa::sketch::plot(&sketch, &PlotOptions::default())` and save the result with `to_png()`. It draws the sketch as it is, solved or not. The manual's sketch pictures come from `crates/swx-qa/examples/plate_sketch_plot.rs`, which does exactly this:

TERMINAL

```
cargo run -p swx-qa --example plate_sketch_plot -- <dir> 600x420
```

It writes `sketch-rough.png`, `sketch-shape.png` and `sketch-sized.png` in `<dir>`.

Only distances, radii and diameters are labeled. Other constraints, such as horizontal or fixed, don't show in the picture, and there are no dimension arrows yet.

CHAPTER 8

Parameters and history

> **FOR DEVELOPERS**
>
> This chapter uses the document model, `swx-core`, and the sketcher, `swx-sketch`, from Rust. It needs no OpenCascade.

The document model holds what makes a part parametric: its named parameters and its ordered list of features (Section 4.8, Section 4.10). It doesn't make geometry itself. Its job is to keep the history valid and to work out what must be rebuilt after a change.

8.1 See what a change rebuilds

YOU WILL	Build a four-feature history and ask which features a change affects.
YOU NEED	Saulitwerx built (Chapter 2).
TIME	5 minutes.

1. Run the sample:

TERMINAL

```
cargo run -p swx-core --example feature_history
```

OUTPUT

```
history:
  0  Sketch1
  1  Extrude1
  2  Sketch2
  3  Revolve1
change thickness -> rebuild Extrude1
edit Sketch2     -> rebuild Sketch2, Revolve1
add Extrude2 with depth `height` -> error: unknown parameter `height`
```

2. Read the history. It is the table in Section 4.10: two sketches, an extrude that uses `thickness`, and a revolve that uses `sweep`.
3. Read the two rebuild lines. Changing `thickness` touches only Extrude1, the one feature that uses it. Editing Sketch2 rebuilds Sketch2 and Revolve1, which consumes it; Sketch1 and Extrude1 are left alone.
4. Read the last line. A feature may use only parameters that exist, so adding an extrude whose depth is `height` is refused.

RESULT You saw Saulitwerx decide, from the history alone, the smallest set of features a change needs rebuilt.

8.2 How the sample is built

The program is `crates/swx-core/examples/feature_history.rs`. A `Document` holds parameters and features:

```
let mut doc = Document::new();
doc.set_param("thickness", 5.0);
let plate_sketch = doc.add_feature("Sketch1", FeatureKind::Sketch)?;
doc.add_feature("Extrude1", FeatureKind::Extrude {
    profile: plate_sketch,
    depth: "thickness".into(),
})?;
```

Then two questions give the rebuild lists, in history order:

```
doc.dirty_after_param_change("thickness");   // [Extrude1]
doc.dirty_after_feature_edit(pin_sketch)?;    // [Sketch2, Revolve1]
```

The features that need rebuilding are called *dirty*. Today's feature kinds are `Sketch`, `Extrude` (with a depth parameter) and `Revolve` (with an angle parameter, in degrees). They are records in the history only; making their geometry comes with the part features on the roadmap.

8.3 Name and rename parameters

Every part keeps its parameters in one parameter table. Each row has a name, a value, a unit and an optional comment, and says where the parameter came from. A *model parameter* belongs to a dimension and gets the next free name in the series `d0`, `d1`, `d2`, … A *user parameter* is one you create yourself, with a name you choose, for a value you want to reuse or change in one place.

Names like `d0` are quick to make but say nothing. Renaming `d0` to `thickness` makes the part easier to read, and every feature that uses it follows the new name, so nothing breaks.

YOU WILL	Build a small parameter table, rename a parameter a feature uses, and see which names are refused.
YOU NEED	Saulitwerx built (Chapter 2).
TIME	5 minutes.

1. Run the sample:

TERMINAL

```
cargo run -p swx-core --example parameter_table
```

OUTPUT

```
model parameters: d0, d1, d2
after renaming d0 to thickness:
  name       kind   value  unit  comment
  thickness  model      5  mm
  d1         model     40  mm
  d2         model     10  mm
  clearance  user     0.2  mm    gap around the pin
Extrude1 depth reads `thickness`
next model parameter: d3
add `2nd_hole` -> error: invalid parameter name `2nd_hole`: must start with a letter or `_`
and contain only letters, digits and `_`
rename d1 to thickness -> error: a parameter named `thickness` already exists
```

2. Read the first line. Three dimensions of a plate (its thickness, width and hole size) were given the model names `d0`, `d1` and `d2`, in the order they were made.
3. Read the table. `d0` is now called `thickness`; it is still a model parameter. `clearance` is a user parameter, with a comment saying what it is for. The rows stay in the order they were made.
4. Read the line about Extrude1. The extrude was made with depth `d0`. After the rename it reads `thickness`, with no change to the extrude itself.
5. Read `next model parameter: d3`. The name `d0` became free when it was renamed, but it is not handed out again. A model name always means one parameter, for the life of the part.
6. Read the last two lines. A name may not start with a digit, and two parameters may not share a name. In both cases the table is left as it was.

RESULT You have a parameter table with meaningful names, and a feature that kept working through a rename.

> **TIP Name what you will change**
>
> Rename the parameters you expect to edit later, such as a thickness or a hole size. Leave the rest as `d0`, `d1`, …

How the sample is built

The program is `crates/swx-core/examples/parameter_table.rs`. Each call works on the document:

```
let depth = doc.add_model_param(5.0, "mm");        // "d0"
doc.add_user_param("clearance", 0.2, "mm")?;
doc.set_param_comment("clearance", "gap around the pin")?;
doc.rename_param(&depth, "thickness")?;
for p in doc.parameters() { /* name, kind, value, unit, comment */ }
```

The unit is free text, such as `"mm"` or `"deg"`, and may be empty; Saulitwerx doesn't check it yet. `set_param`, used in the history sample above, changes a parameter's value; given a new name, it makes a user parameter with no unit. Section 9.9 lists every call.

In Saulitwerx today: parameters hold plain numbers. You can work out an expression such as `d1 / 2 + 5 mm` (Section 8.5), but you can't yet store one in the table.

8.4 Drive a sketch with parameters

A sketch dimension can be tied to a parameter, so the parameter table becomes the place where you change the part's sizes. Each tied dimension makes its own model parameter, `d0`, `d1`, … in the order you add them. Change the parameter, and the dimension follows the next time you apply the table and solve. The tie is called a *binding*.

YOU WILL	Bind a rectangle's width and height to parameters, rename one, change it, and see the sketch follow.
YOU NEED	Saulitwerx built (Chapter 2). OpenCascade is not needed.
TIME	5 minutes.

1. Run the sample:

TERMINAL

```
cargo run -p swx-sketch --example driven_plate
```

OUTPUT

```
dimension 40 mm -> parameter d0
dimension 25 mm -> parameter d1
solved: 40.00 × 25.00 mm
renamed d0 to width (1 dimension follows)
width = 60: 60.00 × 25.00 mm
rename d1 without rename_bound -> error: a sketch dimension is bound to unknown parameter `d1`
height = 0 -> error: value must be > 0, got 0
sketch still 60.00 × 25.00 mm
```

2. Read the first two lines. The width dimension, 40 mm, made parameter `d0`; the height, 25 mm, made `d1`.
3. Read `renamed d0 to width`. The parameter got a meaningful name, and the sketch was told, so its width dimension now reads `width`.
4. Read `width = 60`. Setting `width` to 60 and applying the table made the rectangle 60 mm wide. The height stayed 25 mm.
5. Read the last three lines. Renaming `d1` in the table alone left the sketch looking for `d1`, which no longer exists. A height of 0 was refused, and the sketch kept its size.

RESULT A rectangle whose size you change from the parameter table.

How the sample is built

The program is `crates/swx-sketch/examples/driven_plate.rs`. Instead of `add_constraint`, a dimension goes in with `bind_dimension`, which also adds its parameter:

```rust
use swx_sketch::params::{apply_params, bind_dimension, rename_bound};

let (_, name) = bind_dimension(&mut s, &mut doc, Constraint::Distance {
    a: PointRef::Start(bottom), b: PointRef::End(bottom), value: 40.0 })?;
// name == "d0"
doc.rename_param("d0", "width")?;
rename_bound(&mut s, "d0", "width");
doc.set_param("width", 60.0);
apply_params(&mut s, &doc)?;              // copy the values into the sketch
s.solve(SolveOptions::default())?;        // then move the geometry
```

CAUTION **Rename in both places**

The document and the sketch keep their own lists. `doc.rename_param` renames the parameter and the features that use it, but not the sketch's bindings. Call `rename_bound` with the same two names straight after, or `apply_params` fails with `a sketch dimension is bound to unknown parameter`.

TIP

Only distance, radius and diameter can be bound. A diameter's parameter holds the diameter, not the radius. `apply_params` checks every value before it changes anything, so one bad value leaves the whole sketch as it was.

8.5 Work out an expression

An expression lets one size depend on others: a hole that is always half the plate's width plus 5 mm is `d1 / 2 + 5 mm`. Saulitwerx reads the formula, looks up each name, and checks the units as it goes. Adding a length to an angle is an error, but a length times a length is an area.

YOU WILL	Work out six expressions from a small parameter table, and see each error an expression can give.
YOU NEED	Saulitwerx built (Chapter 2).
TIME	5 minutes.

1. Run the sample:

   TERMINAL

   ```
   cargo run -p swx-core --example expressions
   ```

   The table holds `d0` = 5 mm, `d1` = 40 mm, `wall` = 2.5 mm and `angle` = 30 deg.

   OUTPUT

   ```
   d1 / 2 + 5 mm        = 25.00 mm
   (d1 - 2 * wall) / 3  = 11.67 mm
   2 in + d0            = 55.80 mm
   90 deg - angle       = 60.00 deg
   d1 * d0              = 200.00 mm²
   d1 / d0              = 8.00 (unitless)
   d1 + 5 deg
     error: dimension mismatch at byte 3: cannot apply `+` to length and angle
   5 mmm
     error: unknown unit `mmm` at byte 2 (expected mm, cm, m, in, ft, deg or rad)
   d9 * 2
     error: unknown name `d9` at byte 0
   d1 / (d0 - 5 mm)
     error: division by zero at byte 3
   d1 / (2
     error: syntax error at byte 7: expected `)`
   ```

2. Read the first line. Half of 40 mm is 20 mm, plus 5 mm is 25 mm. Multiplication and division come before addition and subtraction, as in school arithmetic; brackets change the order, as in the second line.
3. Read `2 in + d0`. Units can be mixed: 2 inches is 50.8 mm, plus 5 mm. Results are always worked out in millimeters and radians.
4. Read the next three lines. An angle minus an angle is an angle. A length times a length is an area, in mm². A length divided by a length is a plain number with no unit.
5. Read the errors. Each one says what is wrong and where, counting characters from 0: in `d1 + 5 deg`, byte 3 is the `+`.

RESULT You can work out a size from other sizes, in any of seven units, and trust that mixed-up units are caught.

How the sample is built

The program is `crates/swx-core/examples/expressions.rs`. `parse` reads the text once; `eval` works it out, asking a function you supply for each name's value:

```rust
use swx_core::expr::{eval, parse, Quantity, Unit};

let resolve = |name: &str| -> Option<Quantity> {
    let p = doc.parameters().iter().find(|p| p.name == name)?;
    let unit = Unit::from_suffix(&p.unit)?;
    Some(Quantity::new(p.value * unit.factor(), unit.dim()))
};
let q = eval(&parse("d1 / 2 + 5 mm")?, &resolve, Unit::Mm)?;
// q.value == 25.0 (mm), q.dim == Dim::LENGTH
```

The parameter table stores a unit as plain text, so the sample turns it into a `Unit` itself. Section 9.10 lists the rules and every unit.

> **TIP** **Give every number its unit**
>
> `d1 + 5` is an error, because `d1` is a length and `5` is a bare number. Write `d1 + 5 mm`. A bare number is right only where it scales something, as in `d1 / 2`.

8.6 Common mistakes

- **Using a parameter before it exists.** Call `set_param` first; otherwise `add_feature` fails with `unknown parameter`.
- **Naming a parameter badly.** A name starts with a letter or `_` and holds only letters, digits and `_`: `hole_2` works, `2nd_hole` and `hole-2` don't. Names are case-sensitive.
- **Reusing a name.** Every parameter needs its own name, whether it is a model or a user parameter. To swap two names, rename one of them to a spare name first.
- **Renaming a bound parameter in one place only.** Call `rename_bound` after `rename_param` (Section 8.4).
- **Forgetting to apply and solve.** Changing a parameter changes the table only. Call `apply_params`, then `solve`, to move the sketch.
- **Using a feature that comes later.** A feature may use only features already in the history. Otherwise `add_feature` fails with `feature N references feature M which is not earlier in the history`.

PART IV

Reference

Every command, option, operation and setting that exists today, with its default and its allowed values.

CHAPTER 9

Reference

9.1 `swx-qa render`

Draws a part as an isometric, flat-shaded PNG with its sharp edges outlined.

FORM

```
swx-qa render (--fixture <name> | --step <file.step>) -o <out.png> [options]
```

Option	What it does	Default
`--fixture <name>`	Draws a built-in part: `cube`, `cylinder` or `l-bracket`. Give this or `--step`, not both.	—
`--step <file>`	Draws a STEP file. Needs a build with `--features occt`.	—
`-o`, `--output <file>`	The PNG to write. Required. An existing file is replaced.	—
`--size <W>x<H>`	Picture size in pixels. Each side from 1 to 8192.	`800x600`
`--crease <deg>`	Outline edges where the surface bends by more than this many degrees. 0 to 180.	`30`
`-h`, `--help`	Prints the help text and does nothing else.	—

On success it prints `wrote <file> (<W>x<H>, <bytes> bytes)` and exits with status 0. On an error it prints `swx-qa:` and the message (Section 10.2) and exits with status 1, without writing a file.

Built-in part	Shape
`cube`	A 10 mm cube with one corner at the origin.
`cylinder`	Radius 5 mm, height 10 mm, standing on +Z; 32 flat segments around.
`l-bracket`	An L profile with 30 mm legs and 8 mm thickness, 20 mm deep.

9.2 `swx-qa sketch`

Solves a built-in sketch and draws it from above as a PNG, with its dimension values.

FORM

```
swx-qa sketch --fixture <name> -o <out.png> [--size <W>x<H>]
```

Option	What it does	Default
`--fixture <name>`	The built-in sketch to draw: `rectangle` or `triangle-345`. Required.	—
`-o`, `--output <file>`	The PNG to write. Required. An existing file is replaced.	—
`--size <W>x<H>`	Picture size in pixels. Each side from 1 to 8192.	`800x600`
`-h`, `--help`	Prints the help text and does nothing else.	—

Success and errors work as for `render`. `--step` and `--crease` belong to `render` only.

Built-in sketch	Shape
`rectangle`	40 × 25 mm, lower-left corner fixed at the origin; sides horizontal and vertical; bottom and right dimensioned.
`triangle-345`	A right triangle with its right angle fixed at the origin; a horizontal side of 4 and a vertical side of 3, both dimensioned. The long side comes out 5.

In the picture	Drawn as
Lines, circles, arcs	Dark blue lines, 1 pixel wide.
Points, line ends	Red dots, 5 pixels across.
Distance	Its value, in green, beside the middle of the two points, on the side away from the sketch's center.
Radius, diameter	Its value with `R` or `Ø` in front, outside the circle at the upper right, or outside an arc at its middle.
Values	Up to 3 decimals, with trailing zeros dropped: `40`, `12.5`, `R2.125`.

9.3 Build options

Command	What it builds
`cargo test --workspace`	Everything without OpenCascade, and runs every test.
`cargo build --release -p swx-qa --features occt`	The `swx-qa` tool with STEP input, at `target/release/swx-qa`.
`cargo run -p swx-sketch --example plate_sketch`	Runs the sample sketch (Chapter 5).
`cargo run -p swx-sketch --example sketch_dof`	Runs the degree-of-freedom sample (Section 5.3).
`cargo run -p swx-sketch --example centered_plate -- [<width> <height>]`	Runs the centered-hole sample (Section 5.5). Defaults: 40 and 25.
`cargo run -p swx-sketch --example slot_sketch`	Runs the slot sample (Section 5.6).
`cargo run -p swx-qa --example plate_sketch_plot -- <dir> [<W>x<H>]`	Draws the sample sketch in three stages, as PNGs in `<dir>` (Section 7.5).
`cargo run -p swx-qa --example constraint_plots -- <dir> [<W>x<H>]`	Draws the centered-plate and slot sketches as PNGs in `<dir>`.
`cargo run -p swx-kernel --features occt --example plate_with_hole -- <file.step>`	Builds the plate and saves it (Chapter 6). The file defaults to `plate.step`.
`cargo run -p swx-kernel --features occt --example plate_stages -- <dir>`	Saves the block, the cylinder and the plate as three STEP files in `<dir>`.
`cargo run -p swx-core --example feature_history`	Runs the history sample (Chapter 8).
`cargo run -p swx-core --example parameter_table`	Runs the parameter-table sample (Section 8.3).
`cargo run -p swx-sketch --example driven_plate`	Runs the bound-dimension sample (Section 8.4).
`cargo run -p swx-core --example expressions`	Runs the expression sample (Section 8.5).

The `occt` feature switches on OpenCascade. It is off by default, so a plain build needs only Rust.

9.4 Geometry kernel operations

Operation	What it does and what it accepts
`make_box(corner, size)`	A box with its lowest corner at `corner` and sides `size` = [x, y, z]. Sizes > 0.
`make_cylinder(base, r, h)`	A cylinder along +Z with its base circle centered at `base`. Radius and height > 0.

Operation	What it does and what it accepts
`make_sphere(center, r)`	A sphere. Radius > 0.
`union(a, b)`, `cut(a, b)`, `intersect(a, b)`	Boolean operations (Section 4.9). Inputs unchanged.
`volume(s)`, `surface_area(s)`	In mm³ and mm².
`bounding_box(s)`	The smallest box, aligned with the axes, that holds the solid: its `min` and `max` corners.
`face_count(s)`	The number of faces.
`tessellate(s, deflection)`	Breaks the surface into triangles, no farther than `deflection` (mm, > 0) from the true surface.
`export_step(s, path)`	Writes a STEP file in millimeters, replacing any existing file.
`import_step(path)`	Reads everything in a STEP file as one shape.

All positions must be finite numbers; all sizes finite and greater than zero.

9.5 Sketch entities and constraints

Entity	Defined by
`Point`	x, y.
`Line`	A start point and an end point.
`Circle`	A center point and a radius > 0.
`Arc`	A center, a radius > 0, and start and end angles in radians, running counter-clockwise; the end angle must be after the start. Solving can change all five; it keeps the arc's sweep more than 0 and at most one full turn.

Constraint	Rule
`Coincident(p, q)`	Points `p` and `q` are at the same place.
`Horizontal(line)`	The line's two ends are at the same height.
`HorizontalPoints(p, q)`	`p` and `q` are at the same height.
`Vertical(line)`	The line's two ends are directly above each other.
`VerticalPoints(p, q)`	`p` is directly above or below `q`.
`Fixed { point, x, y }`	The point stays at (x, y).
`Distance { a, b, value }`	Points `a` and `b` are `value` apart. Value > 0.
`Radius { geom, value }`	The circle or arc has this radius. Value > 0.
`Diameter { geom, value }`	The circle or arc has this diameter. Value > 0.
`Parallel(l1, l2)`	Lines `l1` and `l2` point the same way (or exactly opposite ways).
`Perpendicular(l1, l2)`	Lines `l1` and `l2` are at 90°.
`Equal(g1, g2)`	Two lines of equal length, or two circles or arcs of equal radius (any mix of circle and arc). A line and a circle can't be made equal.
`Midpoint { point, line }`	The point is halfway between the line's two ends.
`Symmetric { a, b, line }`	Points `a` and `b` are mirror images across the line: the middle of `a` – `b` lies on the line (or on its extension), and `a` – `b` is perpendicular to it.
`Tangent(g1, g2)`	A line and a circle or arc, in either order, touch: the distance from the center to the line (extended without end) equals the radius. Or two circles or arcs touch, from outside or inside.

A point `p` is written `PointRef::Point(id)` for a point entity, `Start(id)` or `End(id)` for a line's ends, `Center(id)` for a circle's or arc's center, or `ArcStart(id)` or `ArcEnd(id)` for the point of an arc at its start or end angle. Every constraint that takes a point accepts all six.

Tangent detail	How it is decided
Line and circle	The circle stays on the side of the line its center was on when the constraint was added.
Two circles	Touching from outside (centers `r1 + r2` apart) or inside (`|r1 − r2|` apart), whichever the drawing was closer to when the constraint was added.
Line end joined to arc end	When a coincident joins one of the line's ends directly to one of the arc's ends, tangent means the line is square to the arc's radius at that shared point. This holds whichever of the two constraints is added first; a join made through a separate point entity is not recognized.
Where they touch	Otherwise anywhere along the line. Tangent alone doesn't say where a line stops; add other constraints for its ends.

Tangency uses the standard distance conditions for lines and circles; see A. Bowyer and J. Woodwark, *A Programmer's Geometry*, chapters 3 and 5.

9.6 Solver settings and report

Setting	Meaning	Default
`tolerance`	Done when the remaining error is at most this.	`1e-10`
`max_iterations`	Give up after this many attempts.	`100`
`initial_damping`	How cautious the first steps are.	`1e-6`

Stop reason	Meaning
`Converged`	Every rule holds, within the tolerance. `converged` is `true`.
`MaxIterations`	The attempts ran out first.
`Stagnated`	No move makes it better: usually rules that conflict.
`NonFinite`	A value became infinite or not a number.

The solver uses the Levenberg–Marquardt method, a standard way to solve many equations at once; see J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., chapter 10.

9.7 Profiles

`swx_sketch::profile::profile_report(&sketch, &options)` finds the regions a sketch encloses and the pieces left open (Section 4.7). `profiles(&sketch, &options)` returns the regions alone. Point entities are ignored.

Setting	Meaning	Default
`weld_tolerance`	Ends of pieces closer than this count as one corner. `None` uses 0.000001 × the diagonal of the box around the sketch. Must be > 0 when given.	`None`

Result	Meaning
`regions`	Every region found. Each has an `outer` loop and a list of `holes`.
`Region::area()`	The region's area: the outer loop's area minus its holes', in mm².
`Loop`	The `edges` around it in order, each a piece's id and whether it is walked backwards (`reversed`), and its `signed_area`.
`signed_area`	Positive for an outer loop, which runs counter-clockwise; negative for a hole, which runs clockwise. Exact for arcs.
`open_edges`	Ids of the pieces in no loop: free-ended pieces, pieces that only link two loops, and pieces of zero length. Smallest id first.

Rule	What happens
Circle	A loop on its own. So is an arc that goes all the way round.
Loop inside a loop	Becomes a hole of the smallest loop around it. A loop inside that hole is a new region (an island), and so on, alternating.
Pieces that cross, overlap or touch away from their ends	An error, `SelfIntersecting`, naming the first two such pieces. No regions are returned.
No closed loop	No regions, and no error.

Loops are found by walking around each corner in angle order, the standard method for the faces of a flat network of lines; see M. de Berg, O. Cheong, M. van Kreveld and M. Overmars, *Computational Geometry*, 3rd ed., §2.2. Areas use Green's theorem (J. Stewart, *Calculus*, 8th ed., §16.4).

9.8 Degree-of-freedom report

`sketch.dof(options)` checks a sketch as it is now (Section 4.6) and returns a `SketchDofReport`. Run it after `solve`. The lower-level `swx_sketch::dof::analyze(&sketch, sketch.params(), options)` returns the `DofReport` alone.

Setting	Meaning	Default
`rank_tolerance`	How independent an equation must be, compared with the largest, to count as a new rule rather than a repeat.	`1e-9`
`residual_tolerance`	A sketch with a repeated rule is `Conflicting`, not `OverConstrained`, when its remaining error is more than this.	`1e-8`

Report field	Meaning
`dof`	Degrees of freedom left: the sketch's numbers minus its independent equations.
`rank`	The number of independent equations.
`redundant`	The numbers of the equations that repeat earlier ones, smallest first, counting from 0 in the order the constraints were added.
`residual_norm`	The remaining error: how far the rules are from being met, overall.
`state`	One of the states below.

State	When
`UnderConstrained`	No repeated rule, and `dof` is more than 0.
`FullyConstrained`	No repeated rule, and `dof` is 0.
`OverConstrained`	At least one repeated rule, and every rule is met.
`Conflicting`	At least one repeated rule, and the remaining error is more than `residual_tolerance`.
`NonFinite`	A value is infinite or not a number. Nothing else was worked out.

SketchDofReport field	Meaning
`report`	The `DofReport` above.
`redundant_constraints`	The constraints that own a repeated equation, as `ConstraintId`s, smallest first, each listed once. Of two rules that repeat each other, the one added later is listed.

`sketch.constraint_of_row(n)` gives the constraint that made equation `n`, or `None` when there is no such equation.

Entity or constraint	Numbers or equations
Point · Line · Circle · Arc	2 · 4 · 3 · 5 numbers (degrees of freedom when free).
Coincident · Fixed · Midpoint · Symmetric	2 equations each.
Every other constraint	1 equation.

The count uses a Householder QR factorization, taking the equations in the order they were added; see G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., §5.2.

9.9 Parameters

A `swx_core::Document` keeps its parameters in creation order (Section 8.3). Every call that can fail leaves the table unchanged when it does.

Call	What it does
`add_model_param(value, unit)`	Makes a model parameter named `d<N>` and returns the name. `N` starts at 0 and only goes up; a name already taken is skipped.
`add_user_param(name, value, unit)`	Makes a user parameter. Fails if the name is invalid or taken.
`rename_param(old, new)`	Renames a parameter and every feature that uses it. The kind is kept. Renaming to the same name does nothing.
`set_param_comment(name, comment)`	Sets the comment. Fails if there is no such parameter.
`set_param(name, value)`	Sets the value. An unknown name makes a user parameter with an empty unit; this name is not checked.
`param(name)`	The value, or `None`.
`parameters()`	Every parameter, in creation order.
`is_valid_param_name(name)`	Whether a name is allowed.

Parameter field	Meaning	Allowed values
`name`	Unique within the part. Case-sensitive.	a letter or `_`, then letters, digits or `_` (ASCII)
`kind`	Where it came from.	`Model`, `User`
`value`	The number.	any number
`unit`	Free text; not checked yet.	any, may be empty
`comment`	A note for people.	any; empty by default

`swx_sketch::params` ties a sketch's dimensions to a document's parameters (Section 8.4). A sketch remembers each binding as a pair of constraint and parameter name; `sketch.bindings()` lists them in the order they were made.

Call	What it does
`bind_dimension(sketch, doc, c)`	Adds dimension `c` (a `Distance`, `Radius` or `Diameter`) to the sketch, and a model parameter with the same value and unit `mm` to the document; returns the constraint id and the parameter's name. A diameter's parameter holds the diameter. On error neither changes.
`apply_params(sketch, doc)`	Copies each bound parameter's current value into its dimension. Checks every value first, so on error nothing changes. Solve afterwards to move the geometry.
`rename_bound(sketch, old, new)`	Points the sketch's bindings from `old` to `new`; returns how many changed. Call it with every `doc.rename_param`.
`sketch.set_dimension_value(id, value)`	Changes one dimension's value directly. Value finite and > 0.

9.10 Expressions

`swx_core::expr` reads and works out expressions (Section 8.5). `parse(text)` turns the text into an `Expr`; `eval(&expr, &resolve, default_length)` works it out, asking `resolve(name)` for each name's value. `expr.names()` lists the names an expression uses, each once, in first-use order. The result is a `Quantity`: a `value` in millimeters and radians and a `dim` (its powers of length and angle). `q.in_unit(unit)` converts the value for display. `default_length` is the unit for showing lengths; it doesn't change the result.

Part	Rule
Numbers	`5`, `2.5`, `.5`, `1e-3`. A bare number has no unit.
Names	A letter or `_`, then letters, digits or `_`, as for parameters.
Operators	`+ - * /`, unary `-`, and `( )`. `*` and `/` before `+` and `-`; left to right otherwise.
Units	Written straight after a number or a `)`: `5 mm`, `(2 + 3) mm`. After a name, a word is not a unit.
`+` and `-`	Both sides must have the same dimension: two lengths, two angles, or two bare numbers.
`*` and `/`	Any dimensions; they multiply or divide. A length times a length is an area.
Limits	At most 1024 pieces and 64 levels of brackets or minus signs.

Unit	Measures	Size
`mm`	length	1 mm
`cm`	length	10 mm
`m`	length	1000 mm
`in`	length	25.4 mm
`ft`	length	304.8 mm
`deg`	angle	π/180 rad
`rad`	angle	1 rad

9.11 File formats and units

Format	Read / write	Notes
STEP (`.step`, `.stp`), ISO 10303-21	Both	Solids only. Written in millimeters.
PNG (`.png`)	Write	Pictures from `swx-qa render` and `swx-qa sketch`.
Saulitwerx part (`.sltwx`)	Not yet	Planned: the part with its full history.

Quantity	Unit
Length	millimeter (mm)
Area, volume	mm^2, mm^3
Angle	degrees for feature parameters such as a revolve's sweep; radians for arc angles in sketches

9.12 Keyboard and mouse

There are no keyboard or mouse controls yet, because there is no app window. They arrive with the desktop app and will be listed here.

PART V

Help

What to do when something goes wrong: symptoms, every error message you can see, common questions and known limitations.

CHAPTER 10

Help

10.1 Troubleshooting

Symptom	What to do
`cargo: command not found`	Rust isn't installed, or this terminal started before it was. Open a new terminal; if that doesn't help, repeat the Rust step in Chapter 2.
`could not find Cargo.toml`	The terminal isn't in the `saulitwerx` folder. Run `cd saulitwerx` and try again.
A build with `--features occt` fails with errors about `.hxx` files or `TK…` libraries	OpenCascade isn't installed. Install the `libocct-…-dev` packages from Chapter 2.
`swx-qa: cannot read … STEP input needs OCCT`	`swx-qa` was built without OpenCascade. Rebuild it: `cargo build --release -p swx-qa --features occt`.
`target/release/swx-qa: No such file or directory`	The tool isn't built yet. Run the build step in Chapter 2.
Rows of `*` and colored text appear when saving or reading STEP	Normal. The geometry engine prints its own report. Saulitwerx's own lines come before and after it.
The picture looks small or blurry when printed	Use a larger `--size`: about 80 pixels per centimeter of printed width.
A curved part is covered in lines	Raise `--crease`, for example to 45.
The numbers in a sketch picture are too small to read	Use a smaller `--size`. Labels have a fixed size in pixels, so they take up more of a small picture.
A sketch solves, but its sizes aren't what you expect	Some of it is still free to move. Check the degrees of freedom (Section 5.3) and add the missing dimensions.
A sketch looks closed, but has no region	Some ends don't meet. The open-edge list names the pieces (Section 5.7). Join every end, line or arc, to its neighbor with coincident (Section 5.6).
An arc flips or bulges the wrong way after solving	An arc runs counterclockwise from `ArcStart` to `ArcEnd`. Check that each line end is joined to the right end of the arc (Section 5.6).
A sketch dimension ignores a new parameter value	Copy the values into the sketch with `apply_params`, then solve again (Section 8.4).
The solver stops with `Stagnated`	Two or more rules contradict each other. Check the degrees of freedom: the report says `Conflicting` and gives the clashing equation's number.

10.2 Error messages

From `swx-qa`

Every message starts with `swx-qa:`. No picture is written when there is an error.

Message	Meaning and fix
`missing command`	Nothing after `swx-qa`. Add `render`, `sketch`, or `--help`.

Message	Meaning and fix
`unknown command `x``	Only `render`, `sketch` and `help` exist.
`unknown option `x``	A misspelled option, or one that belongs to the other command (`--crease` and `--step` are for `render` only). The help text follows the message.
`` `--size` needs a value ``	An option was last on the line without its value. The same goes for the other options.
`missing input: give --fixture <name> or --step <file.step>`	Say what to draw.
`give exactly one of --fixture and --step`	Two inputs were given. Keep one.
`missing input: give --fixture <name>`	`sketch` needs the name of a built-in sketch.
`missing output: give -o <out.png>`	Say where to save the picture.
`unknown fixture `x`; available: cube, cylinder, l-bracket`	Use one of the listed names.
`unknown sketch fixture `x`; available: rectangle, triangle-345`	Use one of the listed names.
`invalid --size `x`: expected <W>x<H>, e.g. 800x600`	Write the size as two whole numbers joined by `x`, with no spaces.
`invalid --size `x`: width and height must be > 0`	Neither side may be 0.
`image size WxH must be between 1x1 and 8192x8192`	Use a smaller size.
`invalid --crease `x`: expected degrees`	Give a number, such as `45`.
`crease angle x must be between 0 and 180 degrees`	Use a value from 0 to 180.
`cannot read f: invalid input: no such file: f`	The STEP file isn't there. Check the name and the folder.
`cannot read f: kernel failure: import_step from f failed`	The file isn't valid STEP. The geometry engine prints the reason just above.
`cannot write f: …`	The picture can't be saved there, for example because the folder doesn't exist.
`mesh is empty`, `mesh has no visible extent`	The STEP file holds nothing that can be drawn.

From the geometry kernel

Message	Meaning and fix
`invalid input: <name> must be finite and > 0, got <v>`	A size, radius or height is zero, negative or not a number.
`invalid input: <name> must have finite coordinates, got …`	A position contains an infinite value or not-a-number.
`invalid input: no such file: f`	The STEP file to read doesn't exist.
`kernel failure: <operation> failed`	OpenCascade couldn't complete the operation. For a cut, check that the tool solid overlaps the part (Chapter 6).

From sketches

Message	Meaning and fix
`geometry values must be finite`	A coordinate, radius or angle is infinite or not a number.
`radius must be > 0, got r`	A circle or arc needs a positive radius.
`value must be > 0, got v`	A distance, radius or diameter constraint needs a positive value.
`unknown geometry N`	The constraint names an entity that isn't in this sketch.
`geometry N has a different kind`	Wrong kind of point or entity, such as `Center` of a line, `ArcStart` of a circle, a circle in parallel, or a line made equal to a circle (Chapter 5).

Message	Meaning and fix
`unknown constraint N`	No constraint with that number is in this sketch.
`constraint N is not a dimension`	Only a distance, radius or diameter has a value to change.
`only distance, radius and diameter can be bound, got …`	`bind_dimension` was given another kind of constraint. Add it with `add_constraint` instead (Section 8.4).
`a sketch dimension is bound to unknown parameter `p``	The parameter was renamed or never existed. After `doc.rename_param`, call `rename_bound` with the same names (Section 8.4).

From profiles

Message	Meaning and fix
`geometry A and B intersect`	Pieces A and B cross, overlap, or touch away from their ends. Move or shorten one so they meet only end to end (Section 5.7).
`weld tolerance must be > 0, got t`	Give a `weld_tolerance` greater than zero, or `None` for the default.

Any sketch message above can also come from profiles, when the sketch itself is at fault.

From the document model

Message	Meaning and fix
`unknown parameter `p``	Set the parameter before a feature uses it. Renaming or commenting on a parameter that doesn't exist also gives this.
`a parameter named `p` already exists`	Every parameter needs its own name. Pick another, or rename the other parameter first (Section 8.3).
`invalid parameter name `p`: must start with a letter or `_` and contain only letters, digits and `_``	Use a name like `hole_2`: no leading digit, no spaces, dashes, dots or accents.
`unknown feature N`	No feature with that number is in the history.
`feature N references feature M which is not earlier in the history`	A feature can use only features made before it.

From expressions

Each message gives the byte where the problem is, counting from 0 at the first character (Section 8.5).

Message	Meaning and fix
`syntax error at byte n: …`	The text isn't a formula: an unexpected character, a missing `)` or number, or an expression too long or too deeply nested. The rest of the message says which.
`unknown unit `u` at byte n (expected mm, cm, m, in, ft, deg or rad)`	A word after a number isn't one of the seven units. Check the spelling.
`unknown name `p` at byte n`	No parameter has that name. Names are case-sensitive.
`dimension mismatch at byte n: cannot apply `+` to length and angle`	Both sides of `+` or `-` must measure the same thing. Give a bare number its unit, as in `d1 + 5 mm`.
`division by zero at byte n`	The part after `/` works out to zero.

10.3 Questions

Is there a version with windows and buttons? Not yet. The desktop app is a later milestone; see **Coming next**.

Can I open files from other CAD programs? Yes, if you save them as STEP first. Saulitwerx doesn't read any program's own native file format, and won't.

Can I save my design and edit it later? Not yet. Today you can save solids as STEP, which keeps the shape but not the history. Saulitwerx part files come later.

Does it cost anything? No. Saulitwerx is free and open source, under the MIT or Apache-2.0 license, at your choice.

Why millimeters? Mechanical design works mostly in millimeters, so it is the default. Choosing other units per part is planned.

10.4 Known limitations

- No desktop app, 3D view, mouse or keyboard controls yet.
- No features that turn sketches into solids yet: no extrude, revolve, fillet or hole. Solids are made from boxes, cylinders and spheres.
- No native part files; work can't be saved with its history.
- `swx-qa sketch` draws only its two built-in sketches. Your own sketches need a short program (Section 7.5). Sketch pictures show no dimension arrows and no marks for constraints such as horizontal.
- Parameters hold plain numbers. Expressions can be worked out (Section 8.5), but a parameter can't be set to one yet.
- Renaming a parameter doesn't update the sketches bound to it by itself; call `rename_bound` too (Section 8.4).
- There are no angle dimensions and no point-to-line distance yet.
- Profiles don't split pieces where they cross; crossing pieces are an error.
- Only built on and tested on Ubuntu Linux.
- An imported STEP file becomes one shape, and can't be edited.

PART VI

Back matter

Glossary, what changed, what comes next, a one-page card, credits and licenses, and the index.

Glossary

Binding
The link between a sketch dimension and the parameter that drives it.

Bounding box
The smallest box, with sides parallel to the axes, that holds a solid.

Boolean operation
Combining two solids by union, cut or intersect.

CAD
Computer-aided design: using a computer to design physical objects.

Coincident
A constraint that puts two points at the same place.

Conflicting
Said of a sketch whose rules contradict each other, so no shape can meet them all.

Constraint
A rule about a sketch's shape, such as "horizontal" or "these points touch".

Cut
A Boolean operation that removes one solid from another.

Degree of freedom (DOF)
One way a piece of a sketch can still move. A free point has two.

Diameter
The width of a circle through its center; twice the radius.

Dimension
A constraint with a number: a distance, radius or diameter.

Dirty
Said of a feature that must be rebuilt after a change.

Edge
A curve where two faces of a solid meet.

Equal
A constraint that makes two lines the same length, or two circles or arcs the same size.

Entity
One piece of sketch geometry: a point, line, circle or arc.

Face
One flat or curved patch of a solid's surface.

Expression
A small formula with units that works out a value, such as `d1 / 2 + 5 mm`.

Feature
One step in building a part, such as a sketch or a cut. Features are kept in the history.

Feature tree
Another name for a part's history.

Fixed
A constraint that keeps a point at exact coordinates.

Fully constrained
A sketch whose constraints and dimensions leave nothing free to move.

Geometry kernel
The part of a CAD program that does the 3D math. Saulitwerx uses OpenCascade.

History
A part's features, in the order they were made.

Intersect
A Boolean operation that keeps only what two solids share.

Island
A region inside a hole of another region.

Isometric
A view from a corner, looking down at an angle, so that three sides of a box show equally.

Loop
A closed chain of sketch pieces, each ending where the next begins.

Midpoint
A constraint that keeps a point halfway along a line.

Model parameter
A parameter that belongs to a dimension, named `d0`, `d1`, … in turn.

Open edge
A sketch piece that isn't part of any closed loop.

OpenCascade (OCCT)
The open-source geometry kernel Saulitwerx uses.

Over-constrained
Said of a sketch with a rule that repeats what other rules already say.

Parallel
A constraint that makes two lines point the same way.

Parameter
A named number that features and dimensions refer to.

Parametric
Built from steps and numbers that can be changed later, with the design rebuilding to match.

Perpendicular
A constraint that makes two lines meet at a right angle.

Pre-alpha
Early, unfinished software: the parts work, but it isn't ready for real use.

Profile
The regions of a sketch: the areas a feature can turn into a solid.

Redundant
Said of a rule that repeats what other rules already say.

Region
A flat area enclosed by an outer loop, minus any holes inside it.

Sketch
A 2D drawing on a plane, made of points, lines, circles and arcs.

Solid
A closed 3D shape with an inside and an outside.

Solver
The calculation that moves sketch geometry until every constraint holds.

Symmetric
A constraint that makes two points mirror images across a line.

STEP
An international standard file format (ISO 10303) for exchanging 3D parts.

Surface area
The total area of a solid's faces.

Tangent
Touching without cutting in: a line and a circle, or two circles, that meet at one point.

Under-constrained
Said of a sketch with at least one degree of freedom left.

Union
A Boolean operation that joins two solids into one.

Unit
What a number measures in, such as millimeters.

User parameter
A parameter you create yourself, with a name you choose.

Volume
How much space a solid fills.

What's new

2026-10-28: dimensions as parameters, expressions, and joined arcs

- **New:** constraints can name an arc's two ends (`ArcStart`, `ArcEnd`), so lines and arcs join into closed outlines. The slot task is rewritten: it now starts from a careless drawing and solves to a fully defined slot (Section 5.6, Section 9.5).
- **New:** a sketch's distance, radius and diameter dimensions can each make a model parameter, `d0`, `d1`, …, and follow it when it changes (Section 8.4).
- **New:** work out expressions with units, such as `d1 / 2 + 5 mm`, with unit checking (Section 8.5, Section 9.10).
- **Pictures:** the slot before and after solving replaces the slot with a gap.
- Help: new error messages for bound dimensions and expressions; troubleshooting and known limitations updated. The limitation on arc ends is gone.

2026-10-26: parameter table, and named conflicts

- **New:** a part's parameters form a table: model parameters named `d0`, `d1`, … in turn, user parameters with names you choose, units and comments. Renaming a parameter updates every feature that uses it (Section 4.8, Section 8.3, Section 9.9).
- **Changed:** the degree-of-freedom report now names the redundant or conflicting constraints, not just equation numbers (Section 5.3, Section 9.8). The matching known limitation is gone.
- Help: two new parameter error messages, and two new common mistakes.

2026-10-25: more constraints, and profiles

- **New:** six constraints: parallel, perpendicular, equal, midpoint, symmetric and tangent (Section 4.4, Section 9.5).
- **New:** keep a hole centered whatever the plate's size (Section 5.5), and draw a slot with tangent, equal-sized ends (Section 5.6).
- **New:** find the regions a sketch encloses, with their holes and areas, and the pieces left open (Section 4.7, Section 5.7, Section 9.7).
- **Pictures:** the centered plate at two widths, and a slot with a gap, ringed by the template.
- Help: profile error messages, two new troubleshooting rows, and known limitations updated, including arcs whose ends can't be constrained.
- Corrected: midpoint and symmetric add two equations each to the degree-of-freedom count.

2026-10-24: degrees of freedom and sketch pictures

- **New:** check how fully a sketch is defined: degrees of freedom left, and whether it is under-, fully or over-constrained, or conflicting (Section 4.6, Section 5.3, Section 9.8).
- **New:** draw a sketch to a PNG with `swx-qa sketch` (Section 7.5, Section 9.2).
- **Pictures:** the sketch sections of **Concepts** and **Sketching** now show the plate sketch, rough and solved.
- Help: new troubleshooting rows and `swx-qa sketch` error messages; known limitations updated.

2026-10-22: first edition

The first edition of this manual. It covers everything on `main` today:

- Installing from source, and a first tutorial that builds, measures, saves and draws a plate with a hole.
- The ideas of parametric CAD: solids, units, sketches, constraints, dimensions, parameters, features and history.
- Sketching with the coincident, horizontal, vertical, fixed, distance, radius and diameter constraints, and the solver.
- Making and combining solids, and saving and reading STEP files.
- Drawing parts with `swx-qa render`.
- The feature history, and finding what a change rebuilds.
- Reference tables, every error message, troubleshooting, and known limitations.

Coming next

These are the next items on the public roadmap. None of them works yet. When one arrives, it moves into the chapters of this manual and into **What's new**.

Soon	What you'll be able to do
Parameters as formulas	Set a parameter to an expression such as `d1 / 2 + 5 mm`, with the table kept in dependency order and loops refused.
Sketch inference	Constraints such as horizontal, tangent and coincident added for you while you draw.
Part features	Extrude, revolve, fillet, chamfer, shell and hole, driven by sketches.
Part files	Save a part with its full history, and open it again.
Desktop app	A window with a 3D view, a model browser, sketch mode and property panels.

Later milestones add assemblies, 2D drawings and exploded views. The full plan is in `docs/ROADMAP.md` in the source code.

Quick reference

BUILD

TERMINAL

```
cargo test --workspace
cargo build --release -p swx-qa \
  --features occt
```

DRAW A PART

TERMINAL

```
swx-qa render --step part.step -o part.png
swx-qa render --fixture cube -o cube.png
  --size 1600x1000   (default 800x600)
  --crease 45     (default 30)
```

DRAW A SKETCH

TERMINAL

```
swx-qa sketch --fixture rectangle -o r.png
  (or triangle-345)
```

SAMPLES

TERMINAL

```
cargo run -p swx-sketch \
  --example plate_sketch
cargo run -p swx-sketch \
  --example sketch_dof
cargo run -p swx-sketch \
  --example centered_plate -- 60 25
cargo run -p swx-sketch \
  --example slot_sketch
cargo run -p swx-kernel --features occt \
  --example plate_with_hole -- plate.step
cargo run -p swx-core \
  --example feature_history
cargo run -p swx-core \
  --example parameter_table
cargo run -p swx-sketch \
  --example driven_plate
cargo run -p swx-core \
  --example expressions
```

SKETCH CONSTRAINTS

Constraint	Keeps
Coincident	two points together
Horizontal	a line level
Vertical	a line upright
Fixed	a point at (x, y)
Distance	two points a length apart
Radius / Diameter	a circle's size
Parallel · Perpendicular	two lines' directions
Equal	two lengths or radii alike
Midpoint	a point mid-line
Symmetric	two points mirrored
Tangent	a line touching a curve

SKETCH STATES

State	Means
Under-constrained	can still move: add rules
Fully constrained	done: 0 DOF left
Over-constrained	a rule repeats others
Conflicting	rules contradict

SOLIDS

Call	Gives
`make_box`	box from corner + sizes
`make_cylinder`	cylinder on +Z
`make_sphere`	sphere
`union` · `cut` · `intersect`	Boolean result
`export_step` · `import_step`	STEP file out / in

UNITS

Millimeters. Angles: degrees in feature parameters, radians in sketch arcs.

Credits and licenses

Saulitwerx is written by its contributors and is licensed under either the MIT license or the Apache License 2.0, at your choice. It is an independent project, not affiliated with or derived from any commercial CAD product.

Component	License	Notes
OpenCascade Technology (OCCT) 7.6	LGPL-2.1 with the OCCT exception	The geometry kernel. Used as a separate, dynamically linked library; not included in Saulitwerx's source.
`png`, `flate2`, `miniz_oxide`, `fdeflate`, `crc32fast`, `adler2`, `simd-adler32`, `bitflags`, `cfg-if`	MIT, Apache-2.0, Zlib or 0BSD	Write the PNG pictures of `swx-qa`.
`cc`, `shlex`, `find-msvc-tools`	MIT or Apache-2.0	Build the bridge to OpenCascade.
Typst 0.15.1	Apache-2.0	Typesets this manual.
Inter 4.0, by The Inter Project Authors	SIL Open Font License 1.1	Headings and labels in this manual. License in `docs/manual/fonts/OFL-Inter.txt`.
Libertinus Serif, DejaVu Sans Mono	SIL Open Font License 1.1; Bitstream Vera license	Body text and code in this manual, as bundled with Typst.

Every picture of a part in this manual was drawn by Saulitwerx's own renderer, `swx-qa render`, and every picture of a sketch by `swx-qa sketch`'s plotter, from geometry made by Saulitwerx.

Index

A

analyze 43
arc ends 25

B

bind_dimension 44
binding 34
boolean operations 15
bounding box 11

C

CAD 2
coincident 12
conflicting 13
constraint 12
constraint id 21
cut 15

D

degree of freedom 13
degree-of-freedom report 20
diameter 13
dimension 13
dirty 33
distance 13
Document 44
dof 43
doubly connected edge list 43

E

edge 11
entities 12
equal 12, 23
expression 15, 45

F

faces 11
feature 15
feature tree 15
fixed 12
fully constrained 13

G

geometry 12
geometry kernel 4, 27

H

history 15
horizontal 12
Householder QR 44

I

intersect 15
isometric 7

L

Levenberg–Marquardt 42
loop 14

M

midpoint 12, 21
model parameter 33

O

occt feature 40
open edges 14
OpenCascade 4
over-constrained 13

P

parallel 12, 22
parameter 15
parameter table 33
parametric 2
perpendicular 12, 22
PointRef 19
profile 14
profile_report 42

R

radius 13
rebuild 33
redundant 14
region 14

S

sketch 12
solid 8, 11
solver 12
STEP 9
surface area 11
swx-qa render 39
swx-qa sketch 39
symmetric 12, 23

T

tangent 12, 23, 42

U

under-constrained 13
union 15
unit 11
user parameter 33

V

vertical 12
volume 11